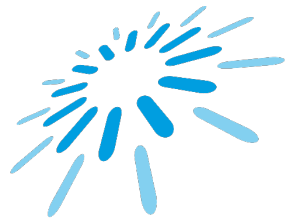




Software Architecture and Design Patterns

Prof. Dr. Jóakim von Kistowski

Lecture 05



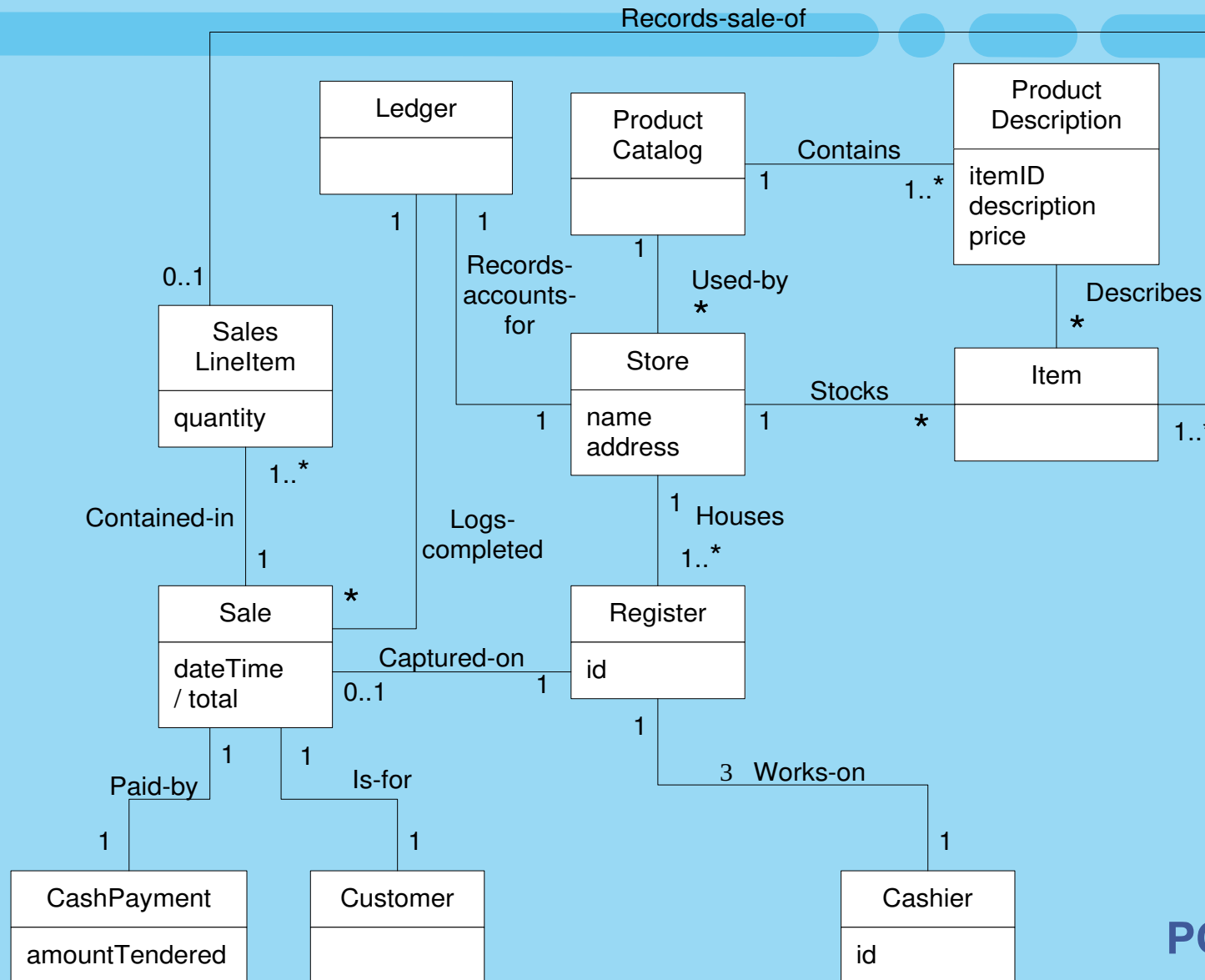
TH Aschaffenburg
university of applied sciences

	Monday	Tuesday	Wednesday	Thursday	Friday
08:00 - 09:30					Vorlesung Software Design
09:45 - 11:15					Übung Software Design
11:30 - 13:00			Lecture Software Design International		
	Lunch Break				
14:00 - 15:30			Exercises Software Design International		
15:45 - 17:15					
17:30 - 19:00					

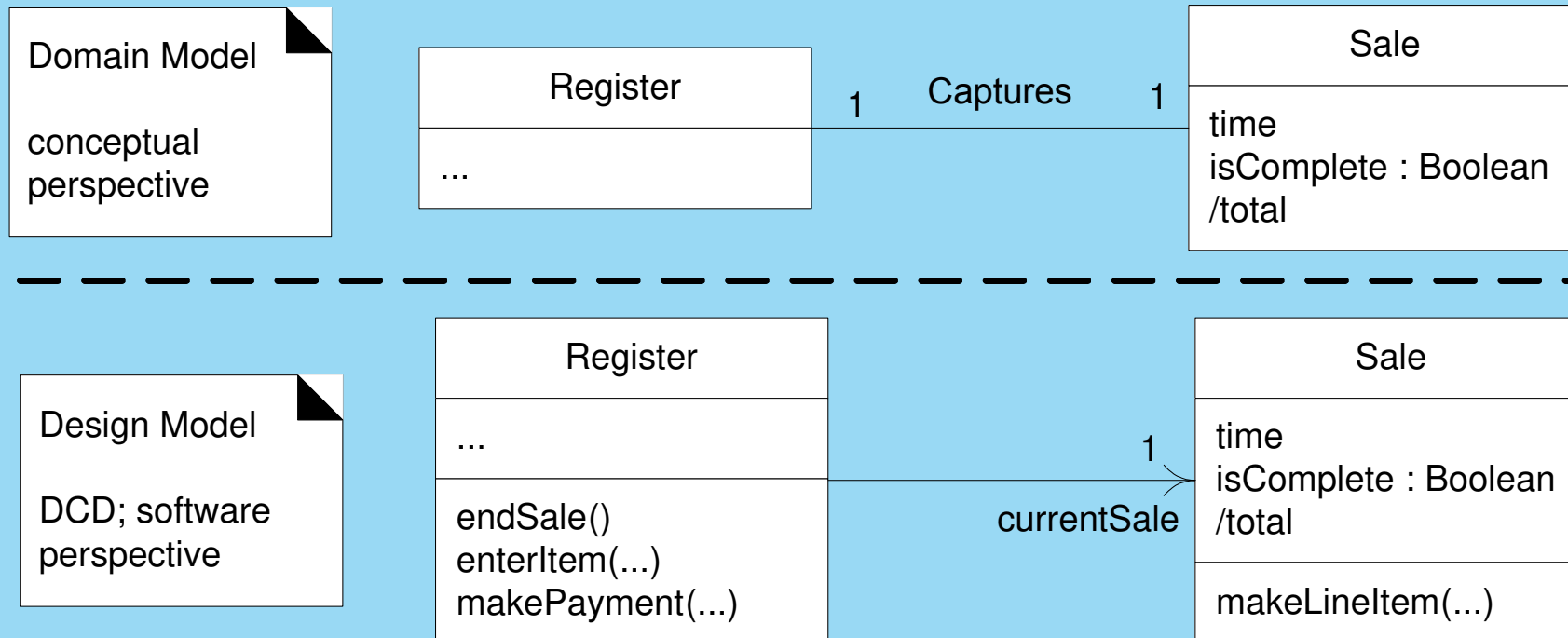
Which UML diagram types do you remember?

What is a domain model?

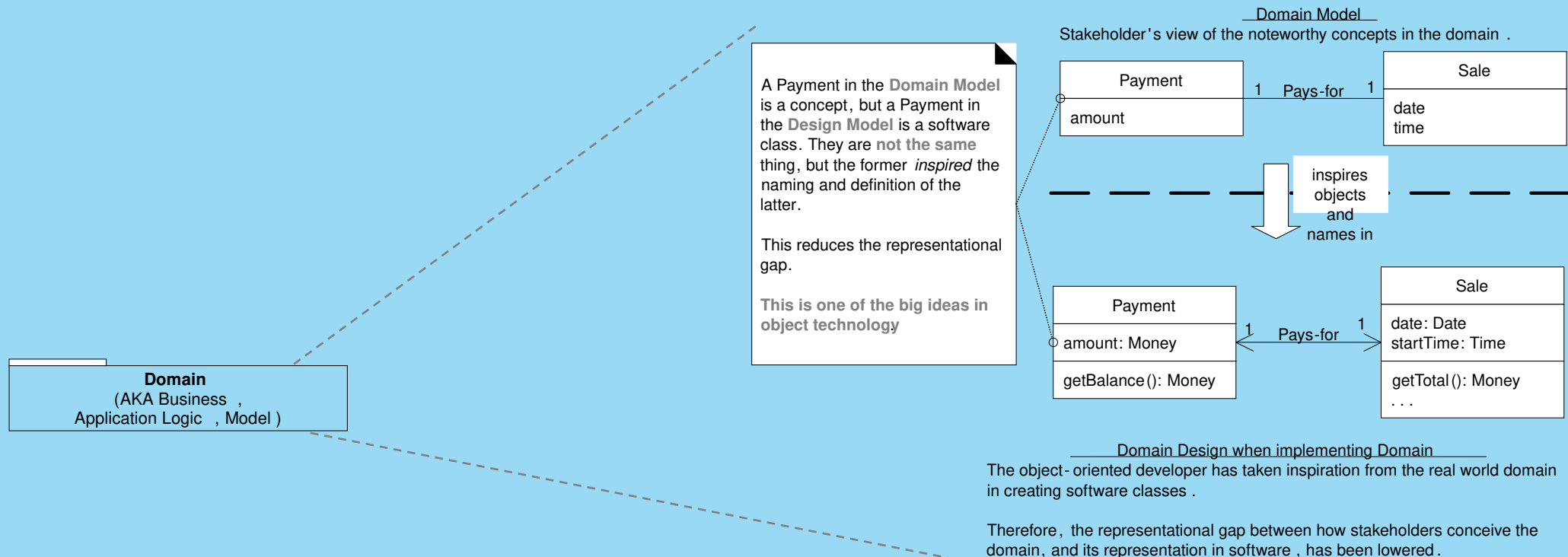
Example Domain Model



Representational Gap



Domain Model vs. Design Model



Object design happens primarily related to implementing the Domain. Nevertheless, the principles in this lecture can be applied in other areas as well.

Object Design (a.k.a. Object-Oriented Design)



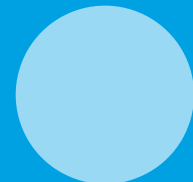
Motivation for Good Object Design

Responsibility-Driven Design (RDD)

GRASP Principles

Further Software Design Principles

Concluding Remarks



Recap: Maintainability ISO 25010

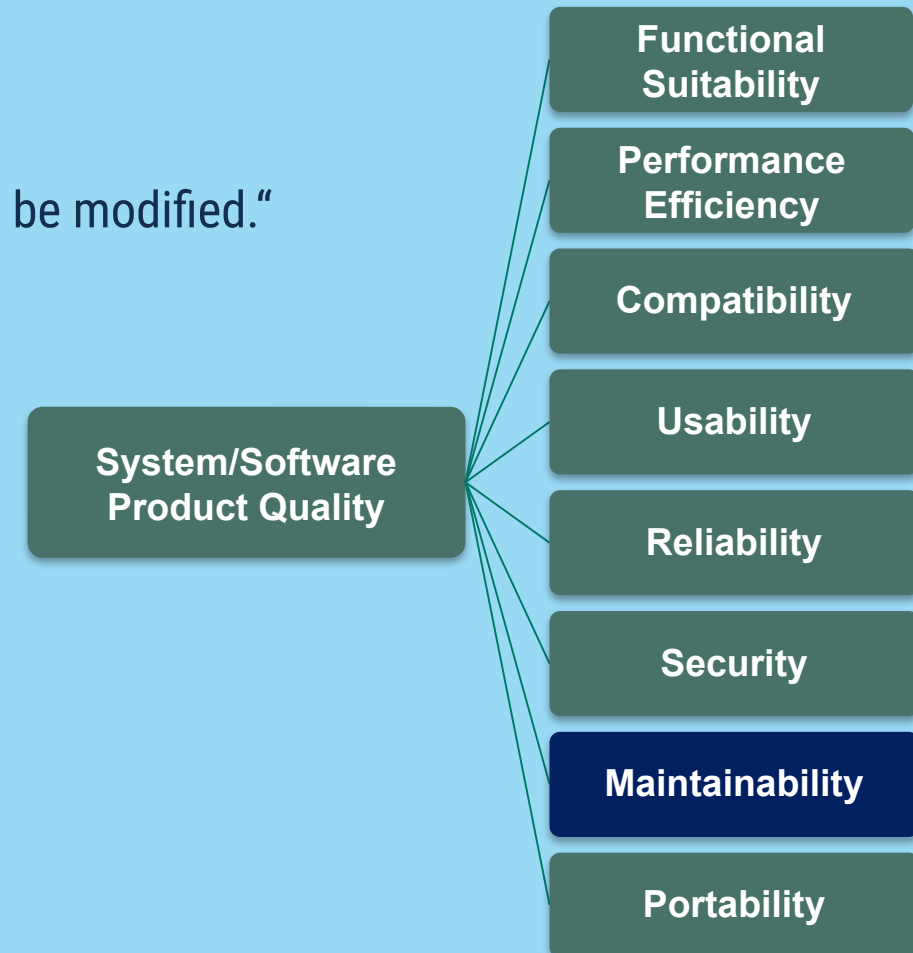
- Increasing pressure to shorten development times
 - Often at the cost of quality
- External quality (users perspective)
 - Usability, reliability, performance,...
- Internal quality (developers perspective)
 - **Maintainability**
- 75% of the total software costs are costs for **maintenance**
 - Correction of bugs / errors
 - Optimization of non-functional properties (e.g., performance)
 - Adaptation to changes in the environment or the requirements



Internal and External Software Quality

Def (Maintainability):

“The degree of effectiveness and efficiency with which the product can be modified.”



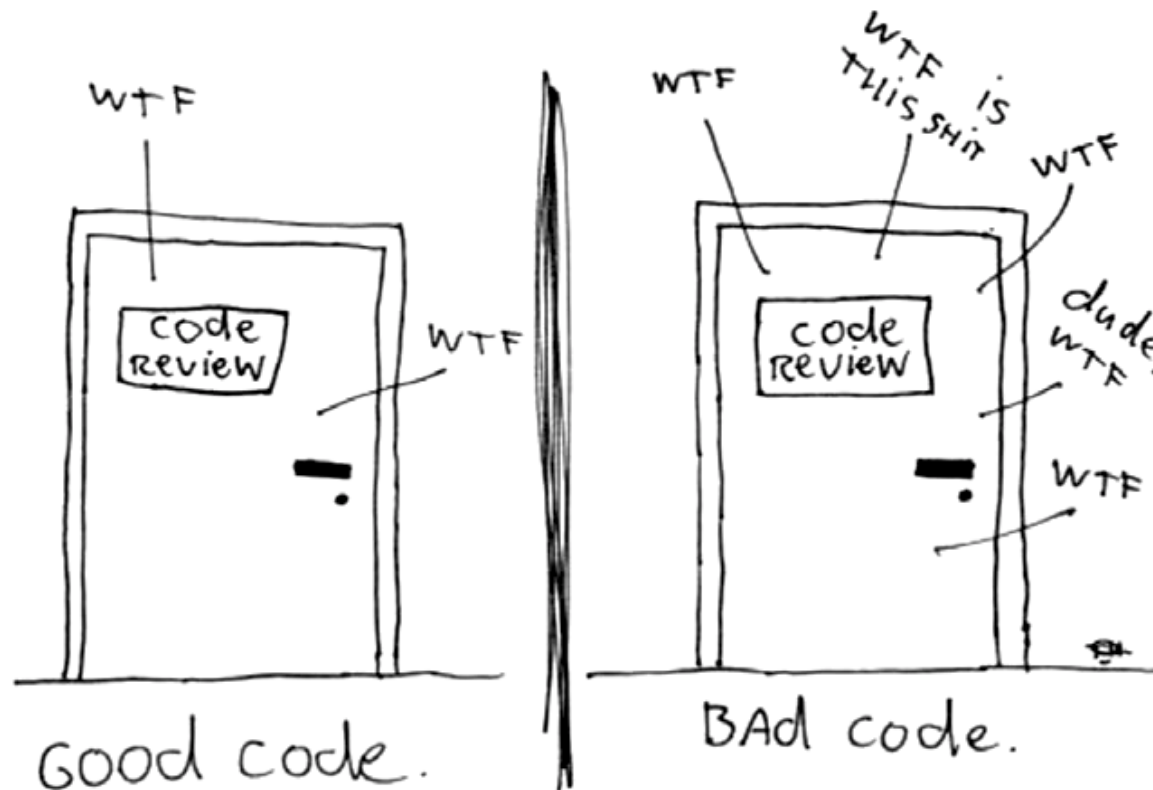
How do we measure maintainability?

Can Code / Design Quality be Measured?



Can Code / Design Quality be Measured?

The ONLY VALID MEASUREMENT
OF CODE QUALITY: WTFs/MINUTE



(c) 2008 Focus Shift

Object Design (a.k.a. Object-Oriented Design)



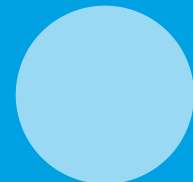
Motivation for Good Object Design

Responsibility-Driven Design (RDD)

GRASP Principles

Further Software Design Principles

Concluding Remarks



Responsibility-Driven Design (RDD)

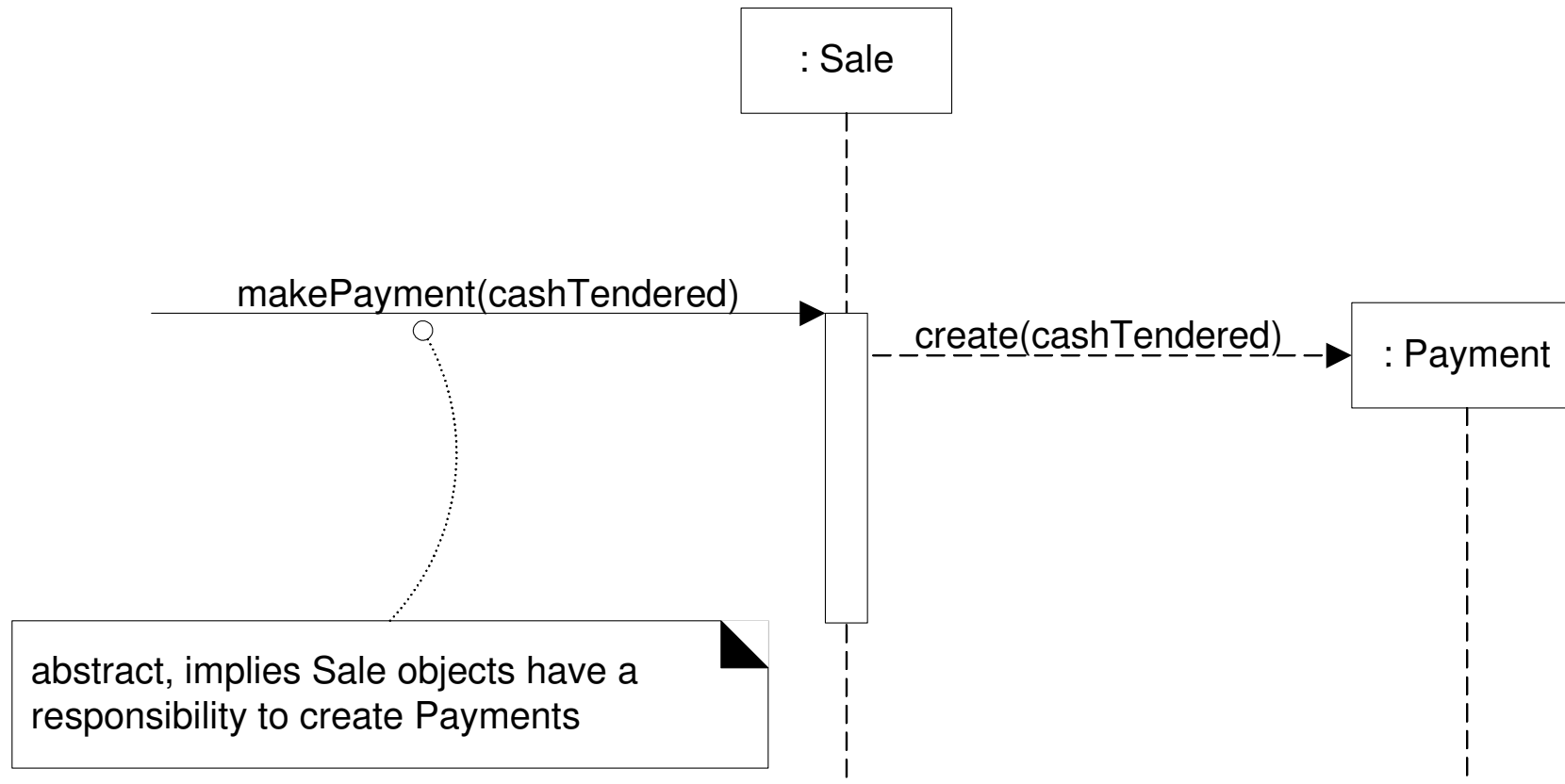
- Software objects are supposed to be similar to objects in the real world
 - A central task in OOD is the **assignment of responsibilities to objects**
- Responsibilities are related to the obligations of an object
 1. **“doing” responsibilities**
 - doing something itself
 - creating an object, doing a calculation (e.g., a Sale creating SaleLineItems)
 - initiating an action in another object
 - controlling and coordinating activities in other objects
 2. **“knowing” responsibilities**
 - knowing about private encapsulated data (e.g., a Sale knowing its total)
 - knowing about related objects
 - knowing about things it can derive or calculate

Object Responsibilities

- Responsibilities are realized using methods
 - that either act alone or **collaborate** with other methods and objects
- However,
a responsibility is **not necessarily the same thing as a method**
- Different **granularities** of responsibilities are possible
 - the responsibility to “provide access to relational databases” may involve dozens of classes and hundreds of methods
 - the responsibility to “create a Sale” may involve just one method

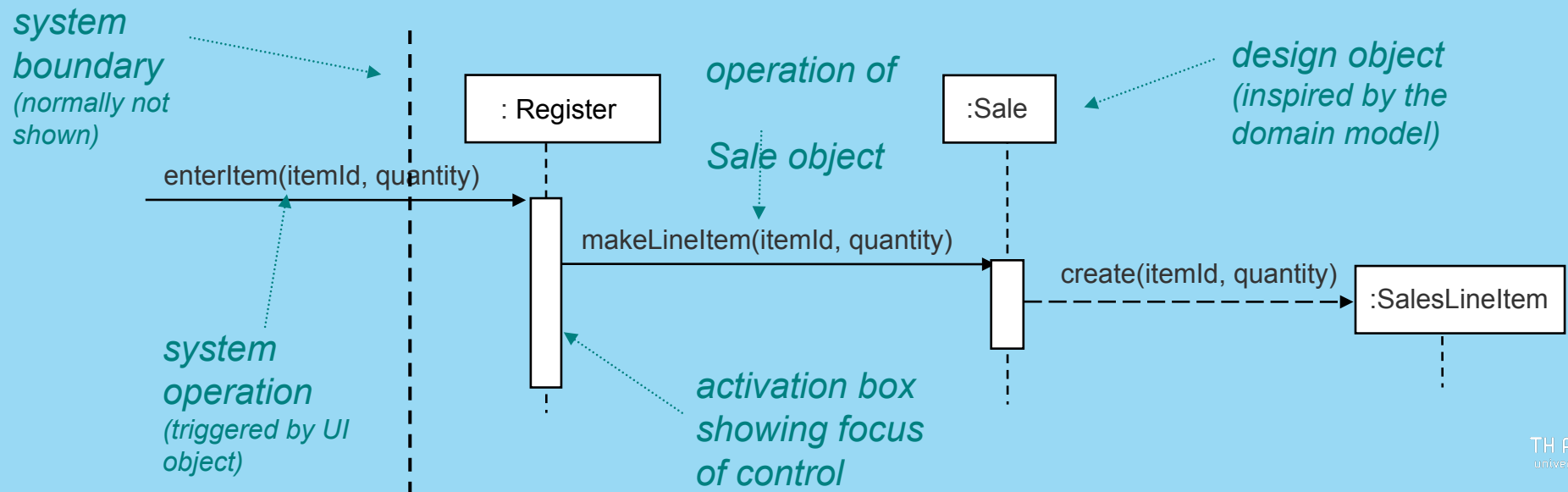
Responsibilities and Methods

Responsibilities are typically assigned using UML interaction diagrams (sequence or communication diagrams)



Example: enterItem

- The message enterItem is sent to an instance of the Register facade
 - the sender (typically the UI) is “outside” of the system
- The following responsibilities are derived from the technical design (interface/operation contracts)
 - the Register instance sends makeLinItem message to a Sale instance
 - the Sale instance creates an instance of a SaleLinItem
 - ...



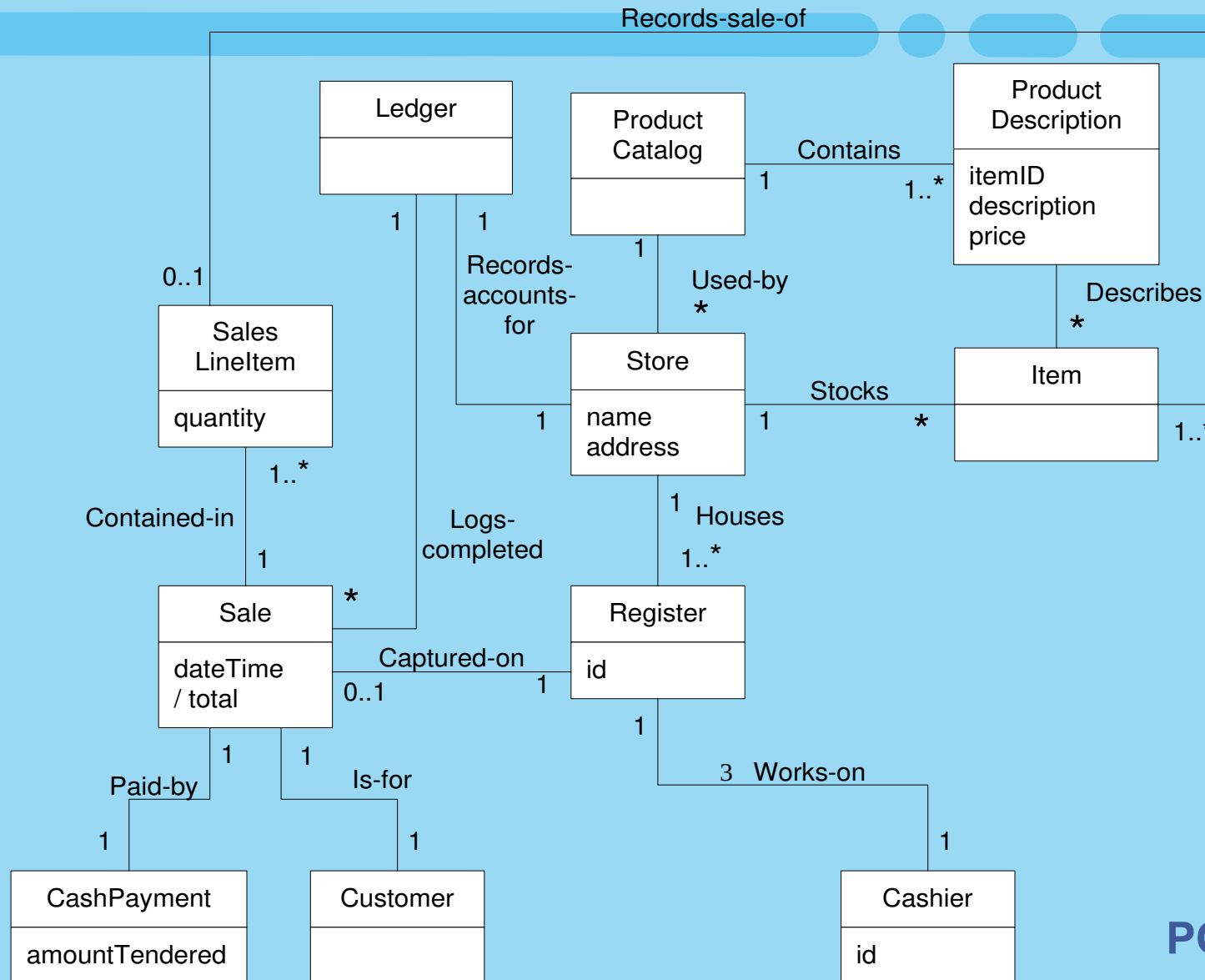
Systematic Approach to Assigning Responsibilities

- Start assigning responsibilities by **clearly stating responsibilities** found in the technical design and use cases
 - e.g., who should be responsible for calculating the grand total of a sale?
- A simple guideline for doing this is the **(Information) Expert Pattern**

Look for the object that has the required information –

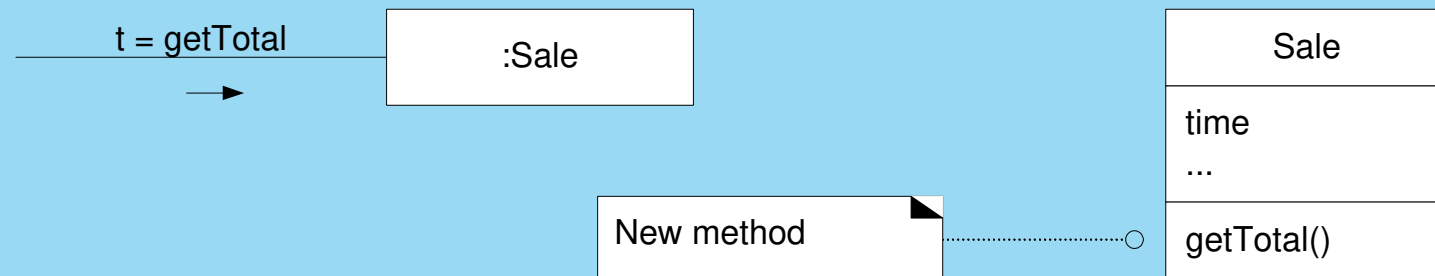
1. If relevant classes already exist in the **design model**, consider them first
2. If not, look at the **domain model** and see if any of the conceptual classes can be added to the design model to fulfill the role
3. If none can be found, consider to use one of the following patterns –
 - Controller or Pure Fabrication (more on these later...)

Example Domain Model



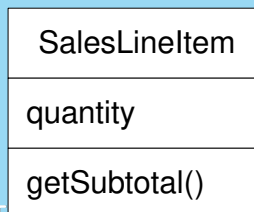
Example: Sale GrandTotal Responsibility (1/5)

- What information is needed to calculate the grand total?
→ the sum of the subtotals for each SalesLineItem instance
- A Sale contains all its SalesLineItems (← domain model)
- → The Sale class is a good candidate for this responsibility
=> assign it the responsibility of knowing its total → add getTotal() method



Example: Sale GrandTotal Responsibility (2/5)

- To fulfill its new responsibility, a Sale object needs the **help of the SalesLineItem objects** it is associated with
 - each SalesLineItem is assigned the responsibility to calculate its own subtotal
 - add a getSubtotal() method to the SalesLineItem design class
- Thus, to calculate the grand total, Sale needs to –
 - send a getSubtotal message to each SalesLineItem
 - and sum the results

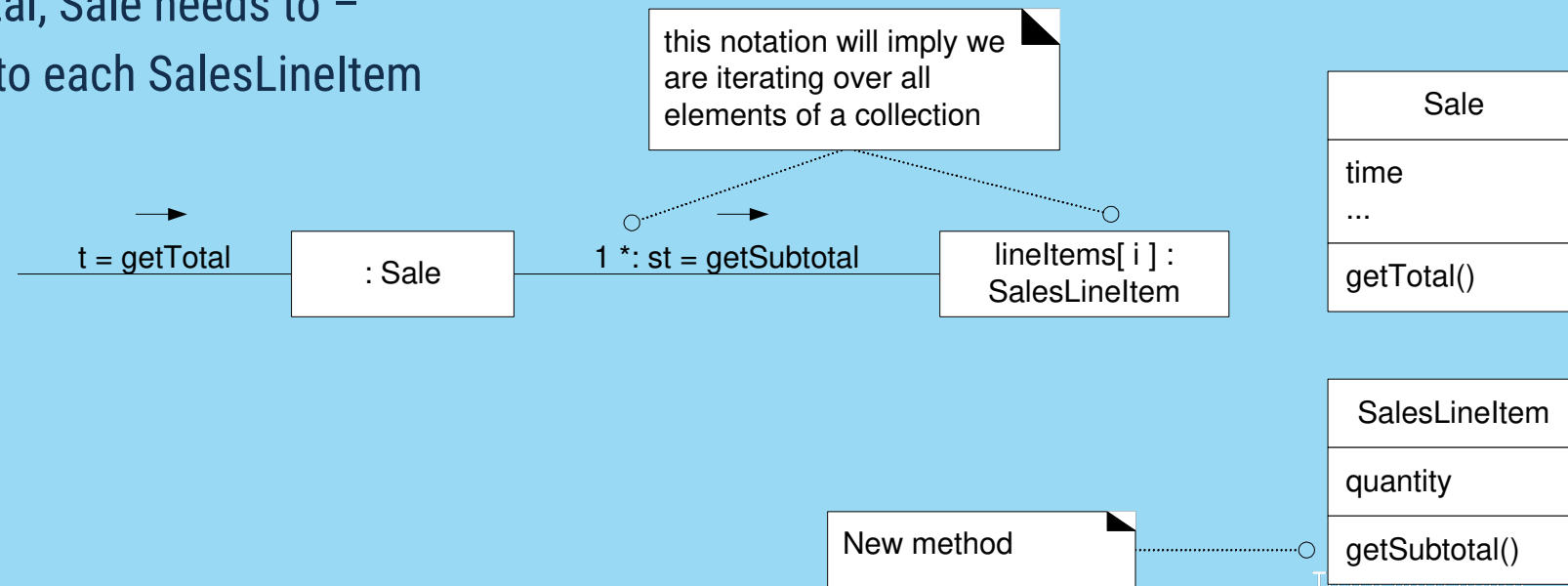


New method



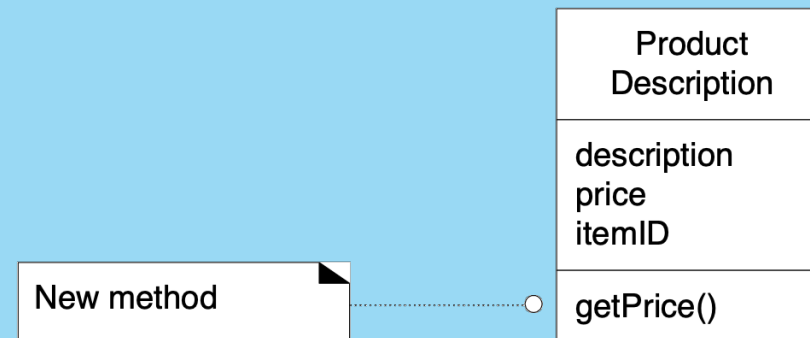
Example: Sale GrandTotal Responsibility (2/5)

- To fulfill its new responsibility, a Sale object needs the **help of the SalesLineItem objects** it is associated with
 - each SalesLineItem is assigned the responsibility to calculate its own subtotal
 - add a getSubtotal() method to the SalesLineItem design class
- Thus, to calculate the grand total, Sale needs to –
 - send a getSubtotal message to each SalesLineItem
 - and sum the results

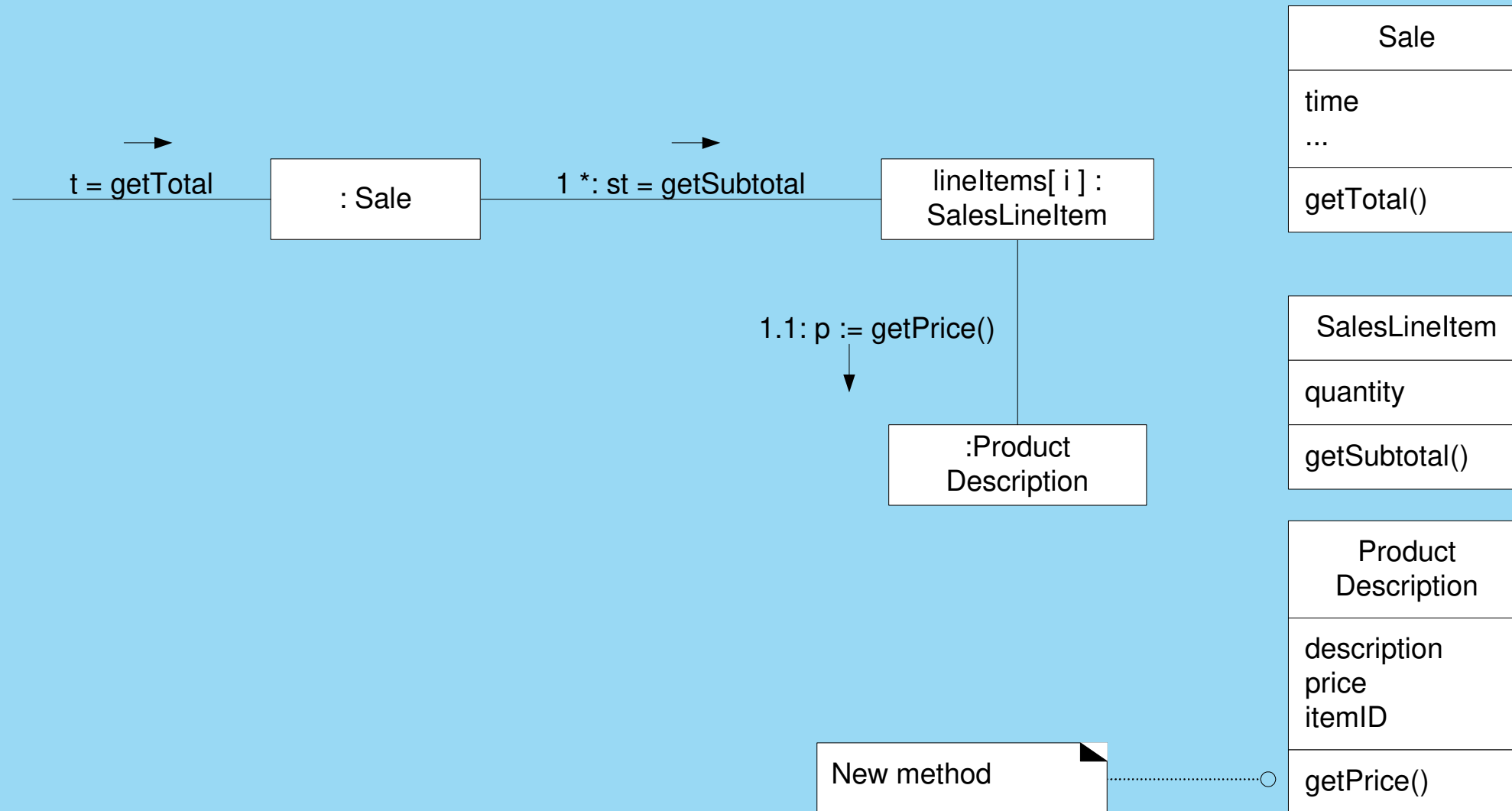


Example: Sale GrandTotal Responsibility (3/5)

- To fulfill its new responsibility (calculate its subtotal), a SalesLineItem object must multiply the number of items by the price per item
 - The **number of items** is stored in its quantity attribute (← domain model)
 - The **price is stored** in the price attribute of the ProductDescription object to which SalesLineItem has a link in the domain model
- To fulfill its responsibility a SalesLineItem object needs the **help of the ProductDescription object** it is linked to
 - → ProductDescription is assigned the responsibility to know the price of the item it describes



Example: Sale GrandTotal Responsibility (4/5)

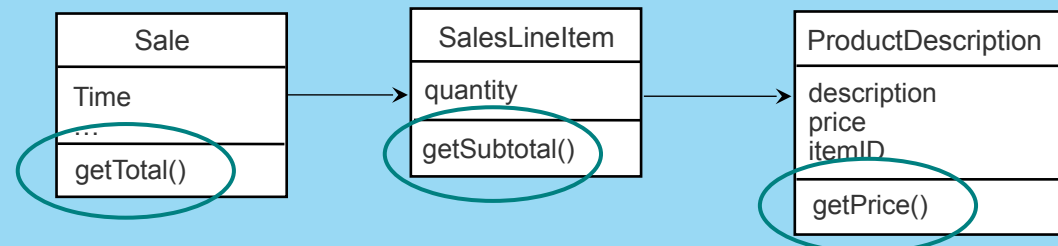


Example: Sale GrandTotal Responsibility (5/5)

- Overall, to fulfill the responsibility of establishing the grand total of a sale, three responsibilities were assigned to three classes

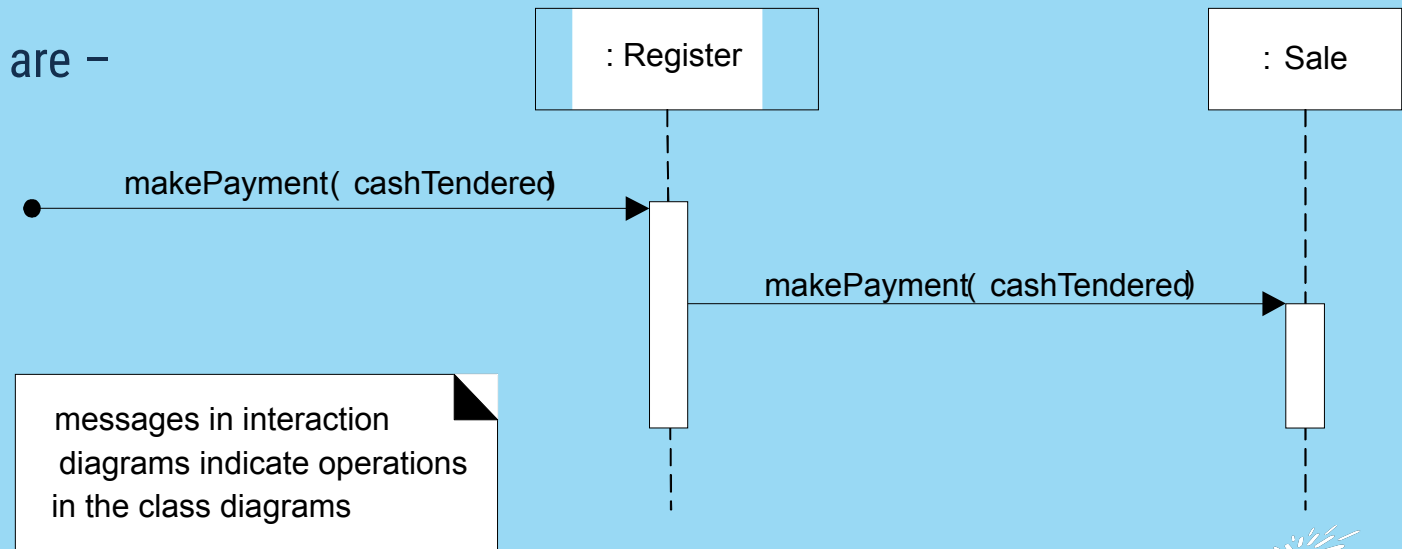
Design Class	Responsibility
Sale	know sales total
SalesLineItem	know line item subtotal
ProductDescription	know product price

- The operation compartments of the corresponding design class diagram can be updated accordingly



Summary: Dynamic Design Model

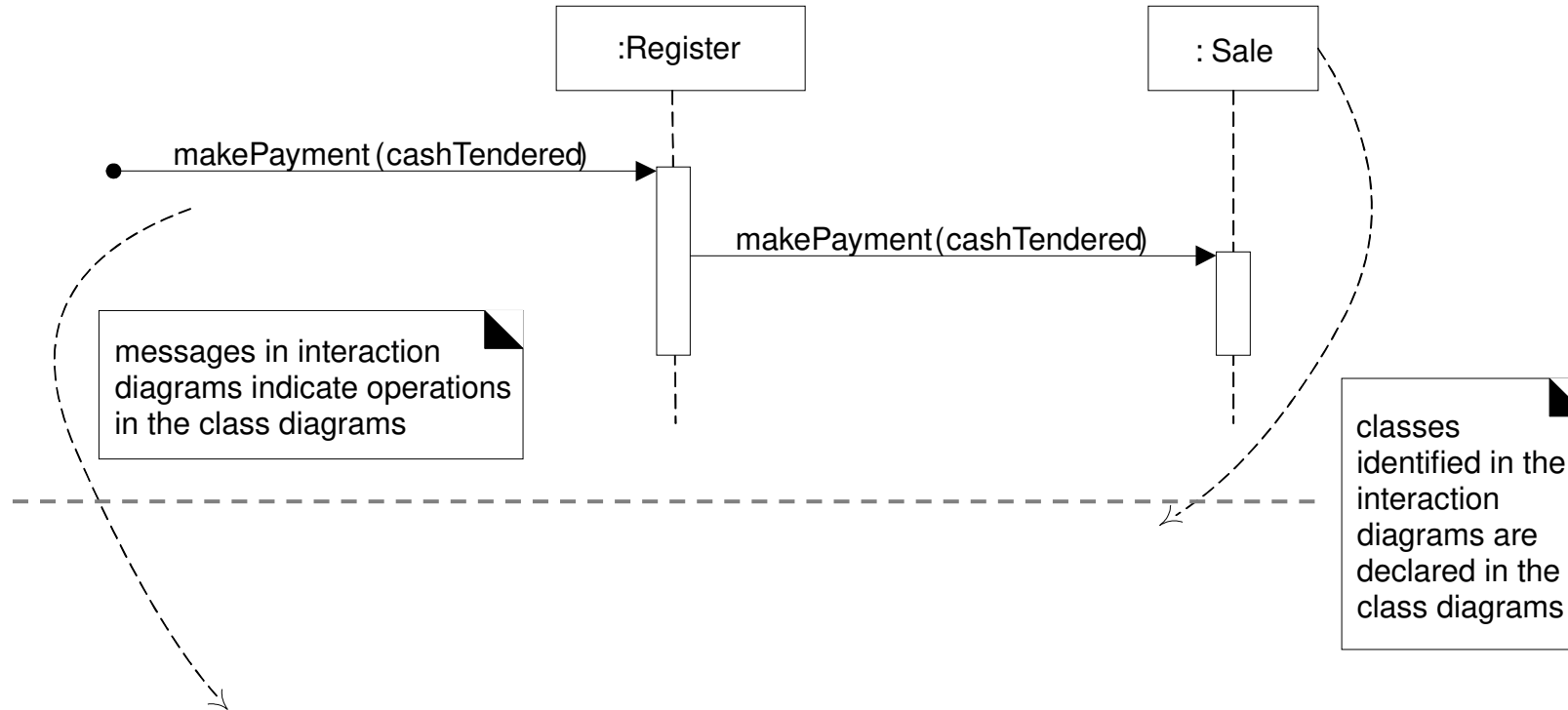
- Typically expressed using **interaction diagrams**
 - UML sequence diagrams and/or communication diagrams
 - more important than class diagrams
 - time invested in their creation is well spent
- The main “ingredients” for their creation are –
 - system operations and their contracts
 - domain model
 - design patterns



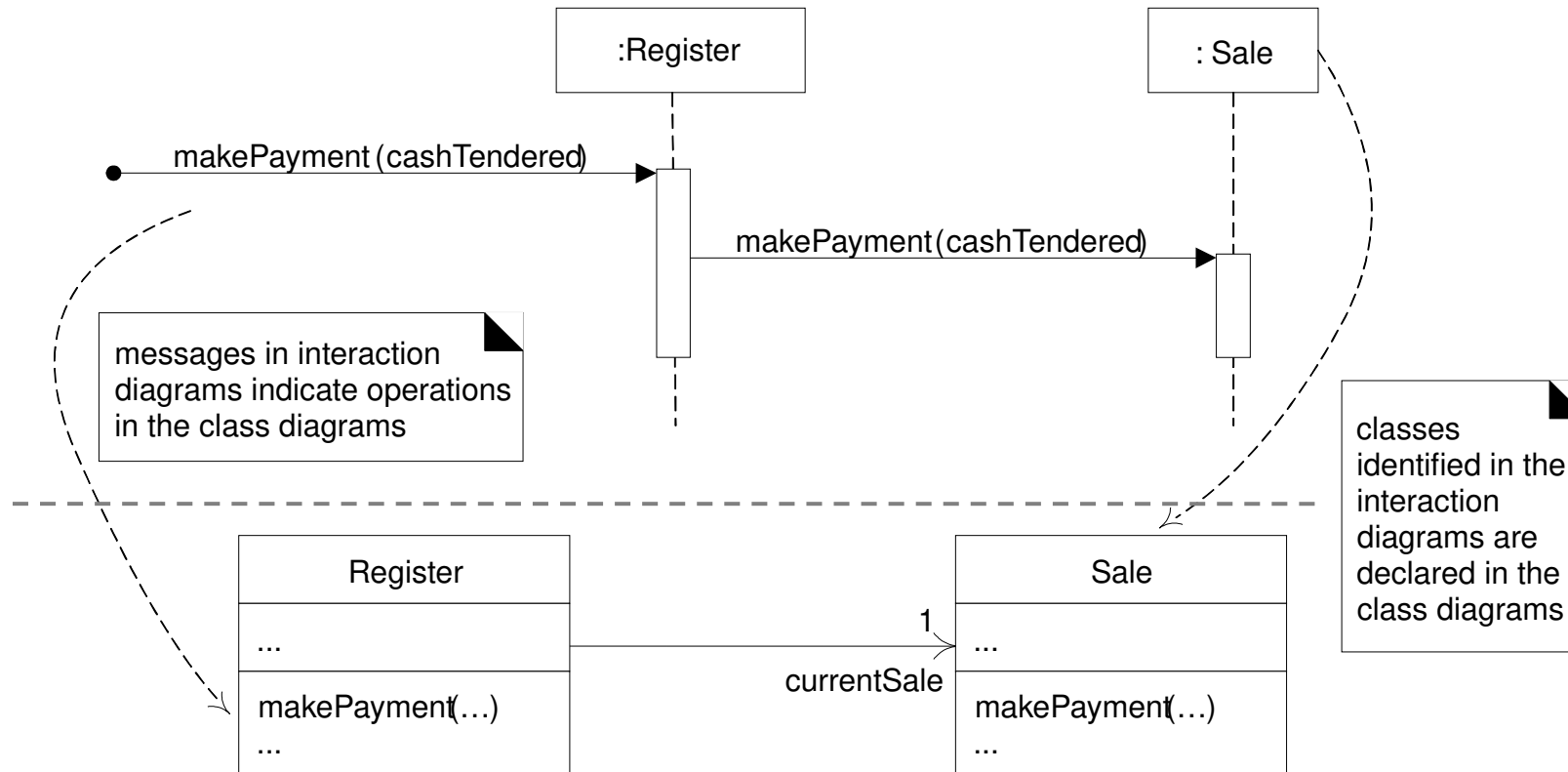
Summary: Static Design Model

- With the completion of interaction diagrams for use-case realizations, it becomes possible to identify **software classes**
 - namely those that participate in the sketched software solution
 - they can be annotated with design details
- A **design class diagram (DCD)** illustrates the specifications for software classes and interfaces in an application
 - in practice, DCDs are developed in parallel with the interaction diagrams
- Typical information included
 - classes, associations, and attributes
 - interfaces with their operations and constants
 - methods
 - attribute type information
 - navigability, dependencies

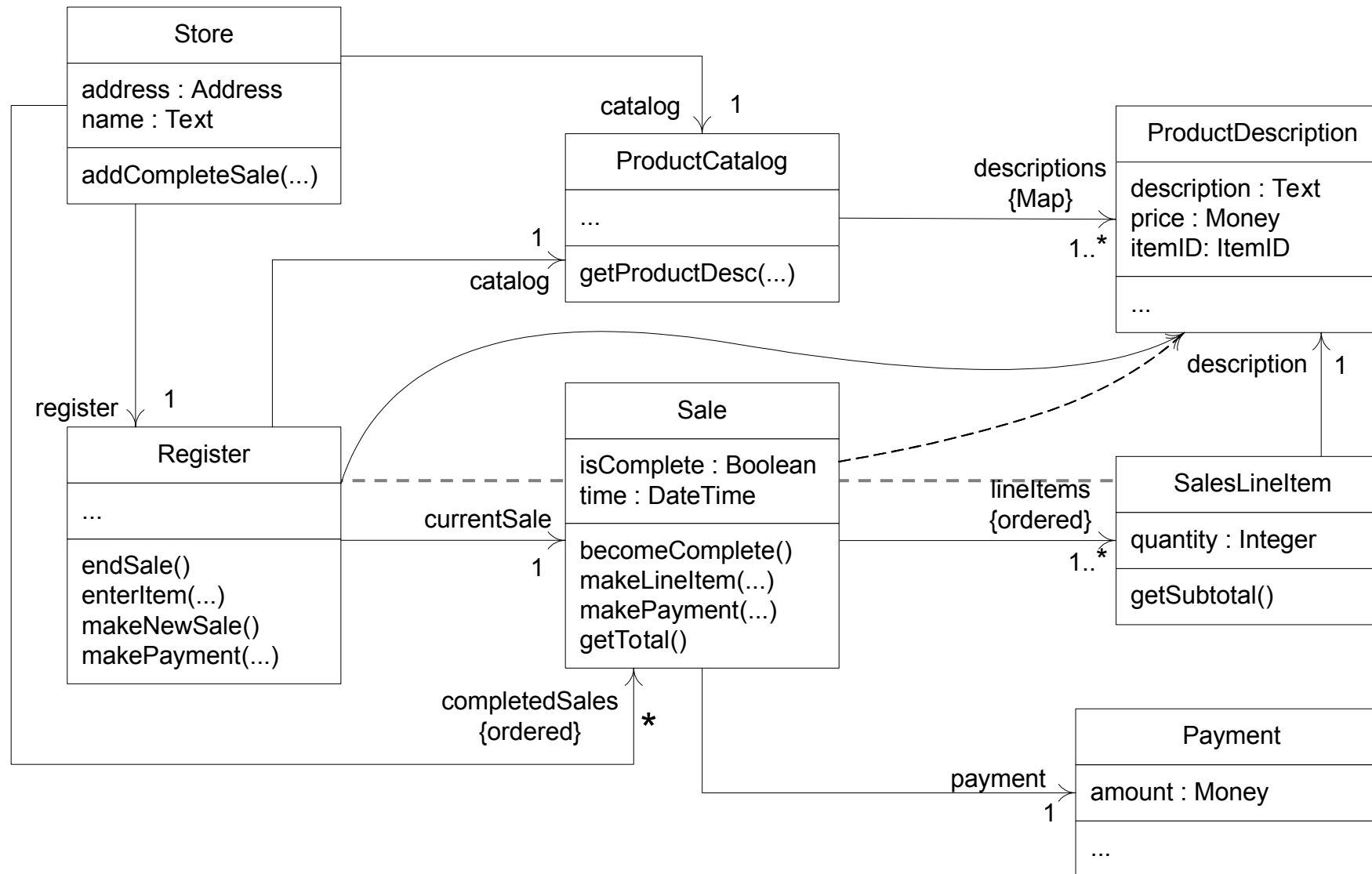
Example: makePayment



Example: makePayment



... Example Design Classes for POS System



Agile Modeling

- Early agile modeling is primarily a means of **communication**
 - initial models will never be perfect
 - as new questions might arise
 - e.g., how do we deal with different SalesLineItems with identical itemIDs?
 - Larman recommends avoiding the use of UML tools at this stage
 - or to use them later for documentation only



Object Design (a.k.a. Object-Oriented Design)



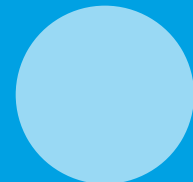
Motivation for Good Object Design

Responsibility-Driven Design (RDD)

GRASP Principles

Further Software Design Principles

Concluding Remarks



Question

Patterns are used in which of the following?

a) software architecture

b) object design

c) implementation/coding

d) all of the mentioned

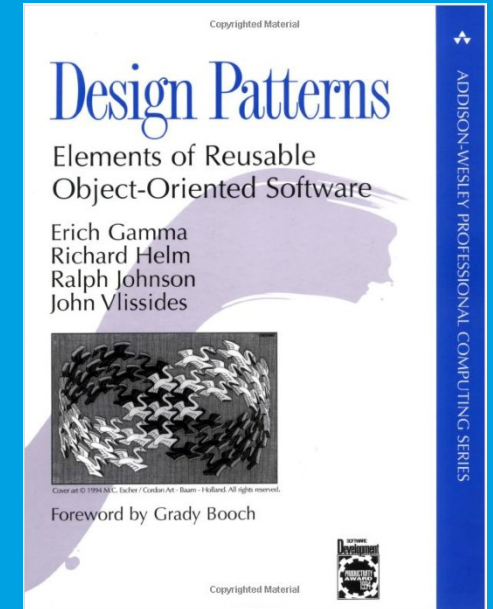
GRASP (Principles)

- Besides the general idea of having a low representational gap between domain and software objects
 - Several guiding principles exist to **help us assign responsibilities**
- **General Responsibility Assignment Software Patterns**
Nine very fundamental, basic principles of object design
 1. Information Expert or just “Expert” (the pattern we just applied)
 2. Creator
 3. Controller
 4. Low Coupling
 5. High Cohesion
 6. Polymorphism
 7. Pure Fabrication
 8. Indirection
 9. Protected Variations

“Luck is the residue of design.”
-- Branch Rickey

Design Patterns

- Idea of patterns in software comes from **Kent Beck** in the mid 1980s
- Design patterns are immutably connected with the famous book by the **Gang of Four** (GoF, cf. [GoF]) from 1994
- 23 patterns commonly referred to as **GoF patterns**
- We first focus on GRASP (principles)



GoF = Erich Gamma + Richard Helm +
Ralph Johnson + John Vlissides

Pattern: “a named description of a well-known problem and solution that can be applied to new contexts; ideally, a pattern advises us on how to apply its solution in varying circumstances and considers the forces and trade-offs” [Larman]

Relationship between GoF and GRASP

- Most OO design patterns, including GoF patterns, can be seen as specializations (special cases) of the nine basic GRASP principles
 - Thus, GRASP can be seen as the essential „alphabet“ of OO design patterns
- Understanding GRASP helps to convey the essence of the huge number of design patterns available in the literature
- Given their fundamental nature, we will typically refer to GRASP patterns as **design principles** to distinguish from classical design patterns (GoF)

Creator (GRASP)

1. Problem

- Who should be responsible for creating a new instance of some class?
-

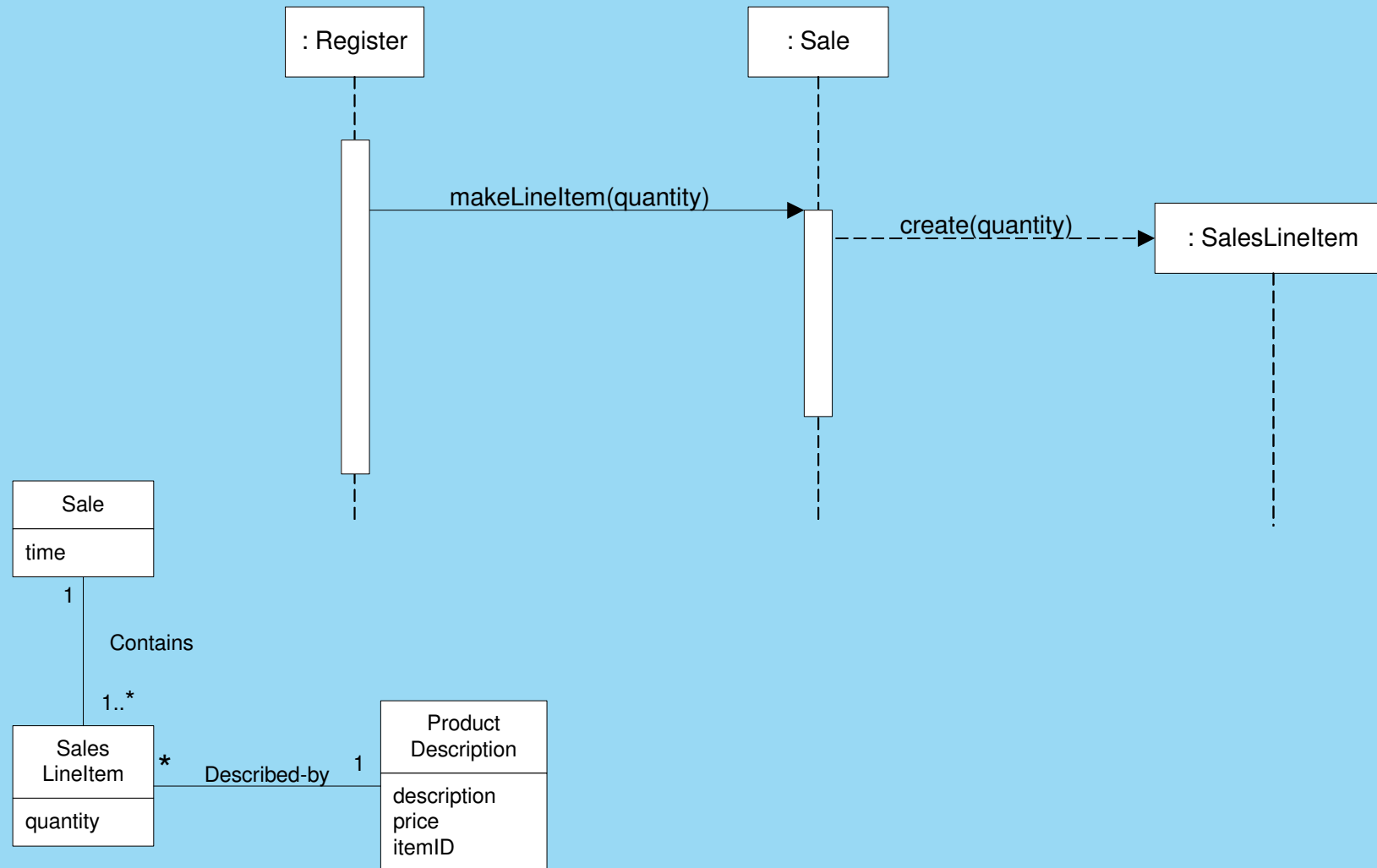
2. Solution

- Assign class B the responsibility to create an instance of class A if one of these is true (the more the better) –
 - B “contains” or compositely aggregates A (i.e., A is a part of B)
 - B records A
 - B closely uses A (i.e., B calls a lot of operations of A)
 - B has the initializing data for A
-

3. Related Patterns / Principles

- GoF Concrete/Abstract Factory

Example: Creator of SalesLineItem?



Low Coupling (GRASP)

Def: Coupling is a measure of how strongly one software element is connected to, has knowledge of, or relies on other software elements (classes, components, subsystems, systems, etc).

1. Problem

- How to support low dependency, low change impact, and foster reuse?

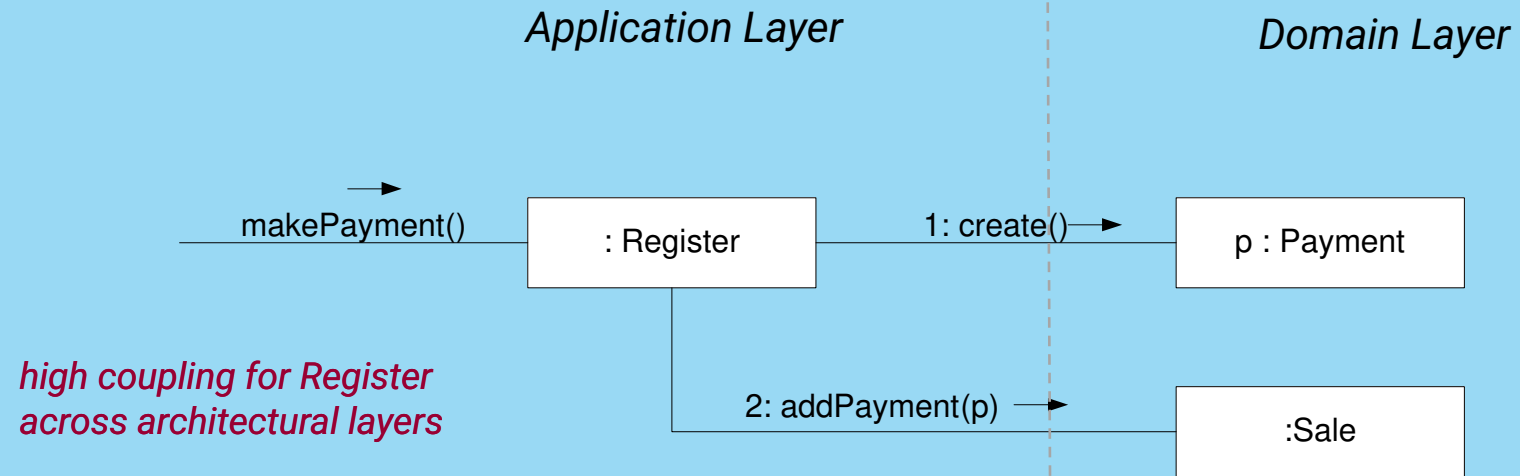
2. Solution

- Assign responsibilities so that coupling remains low!

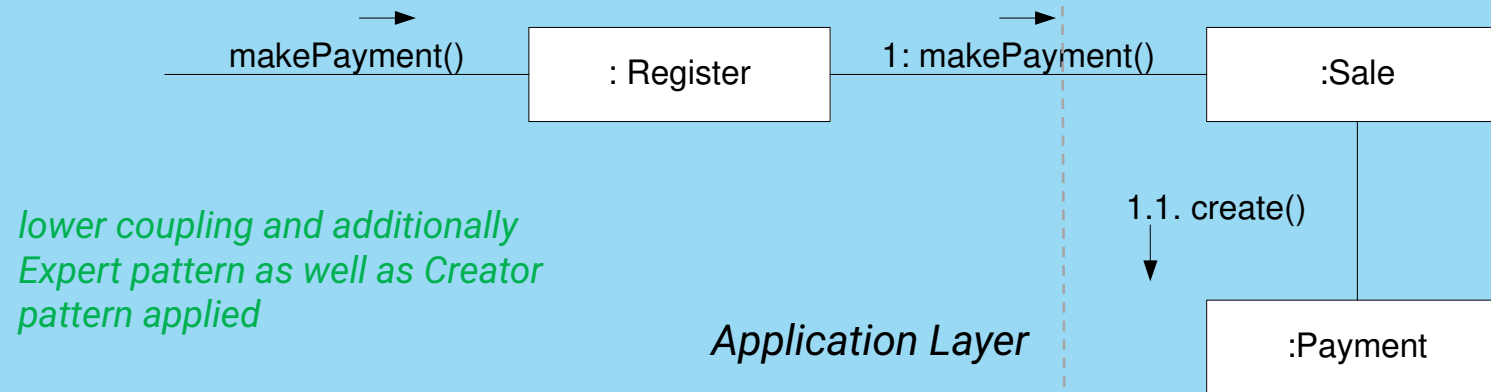
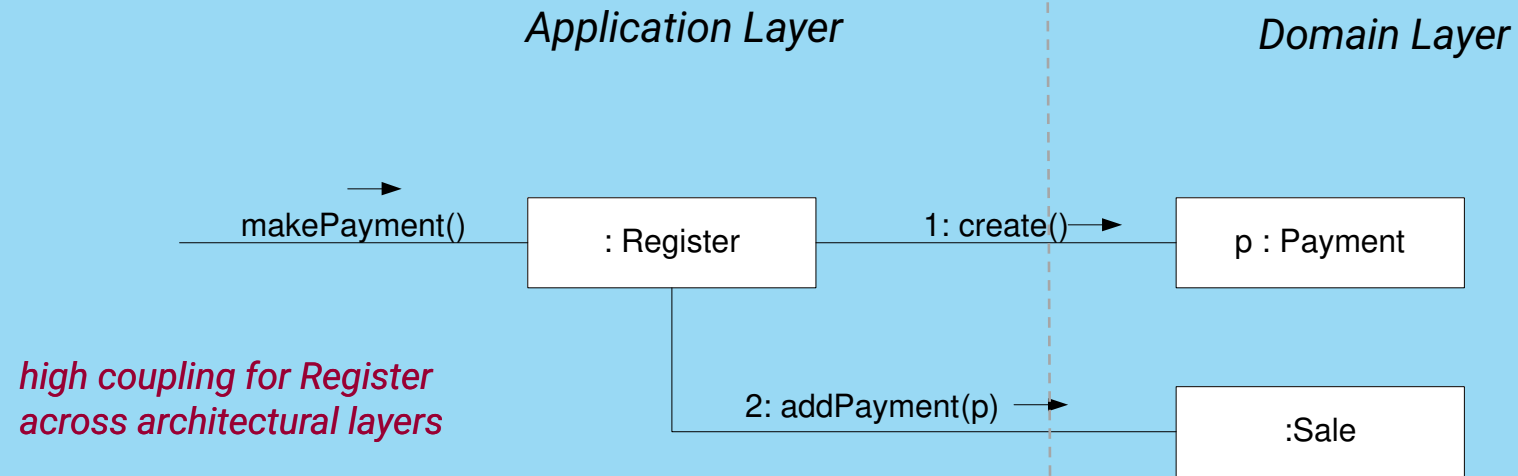
3. Benefits

- Design of classes that are more independent
- Reduced impact of changes
- Simple to understand in isolation
- Easier to reuse

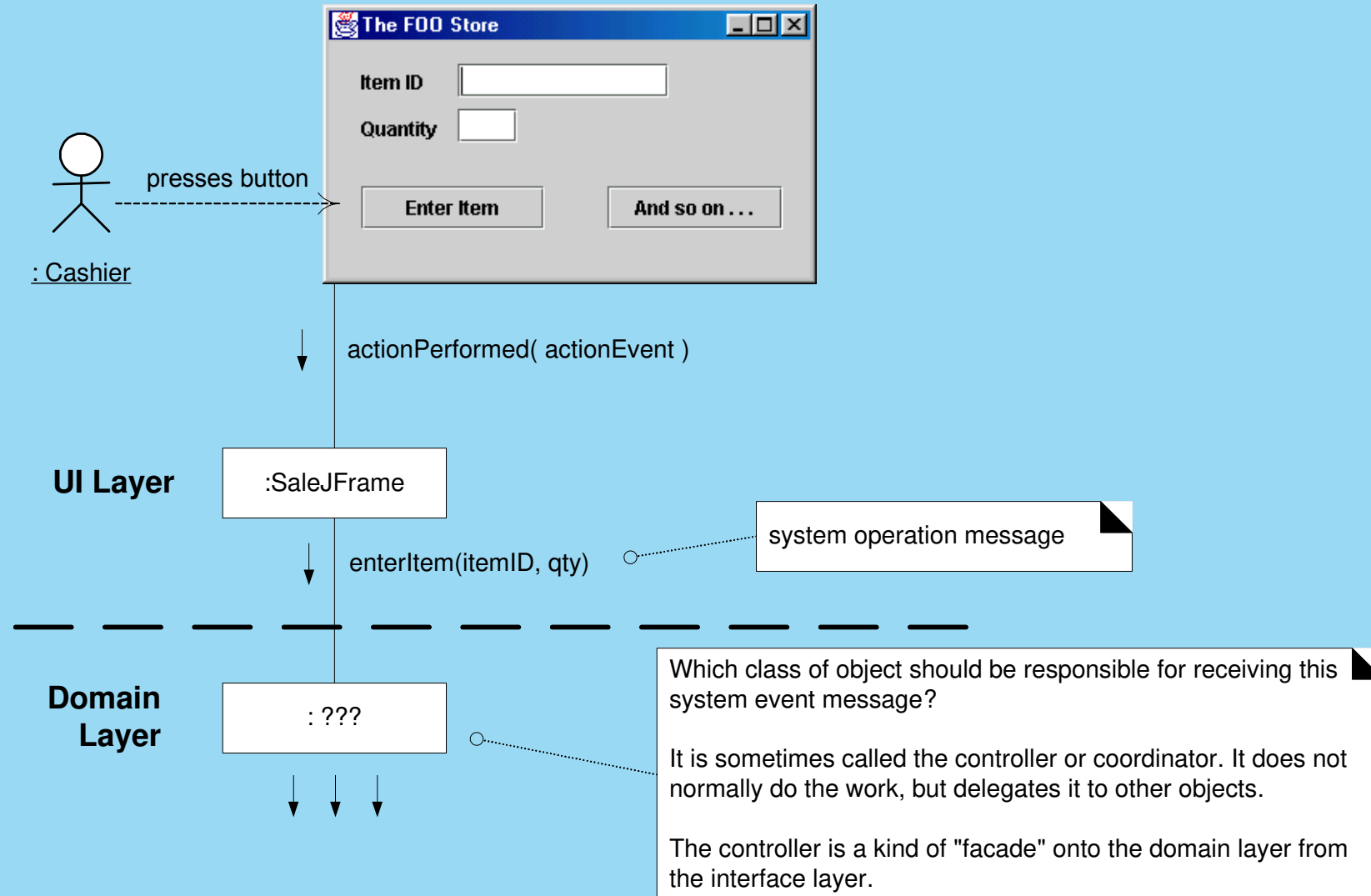
Example: Low Coupling (GRASP)



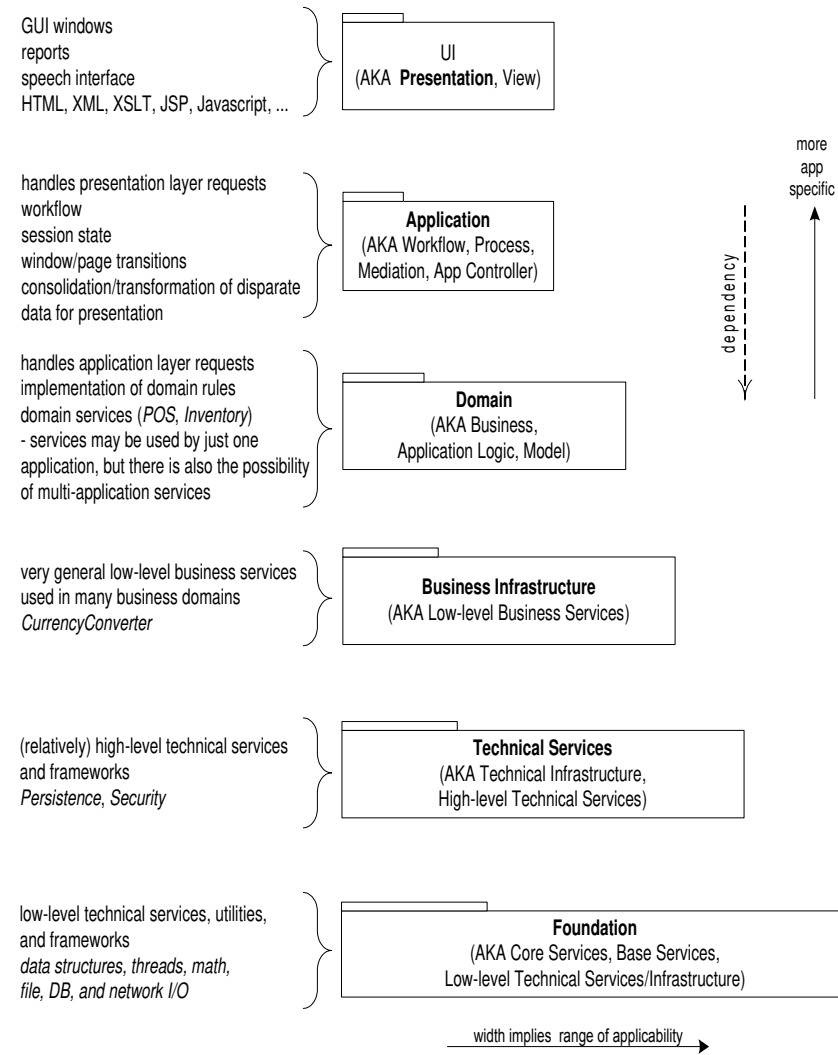
Example: Low Coupling (GRASP)



Motivation: Controller(GRASP)



Layered Architecture



More on layered architectures
in upcoming lectures

Controller (GRASP)

Def: A **controller** is the first object beyond the UI layer that is responsible for receiving or handling a system operation message.

1. Problem:

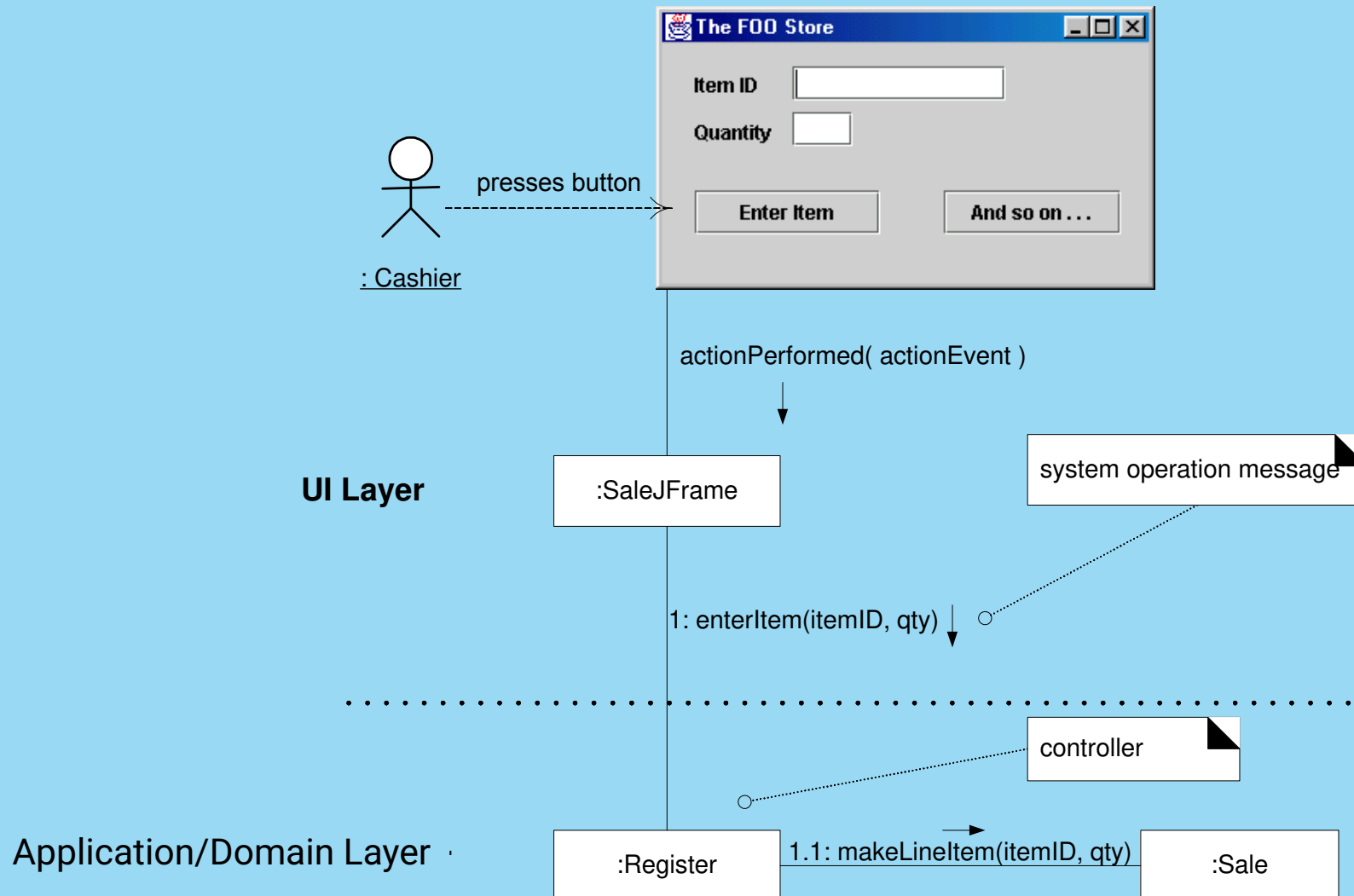
What first object beyond the UI layer receives and coordinates („controls“) a system operation?

2. Solution: Assign the responsibility to one of the following:

1. **System facade controller** ☐ A class representing the overall „system“, a „root object“, a device that the software is running within, or a major subsystem
2. **Use case (or session) controller** ☐ A class representing a use case scenario within which the system event occurs, often named <UseCaseName>Handler, <UseCaseName>Coordinator or <UseCaseName>Session (Informally, a session is an instance of a conversation with an actor)

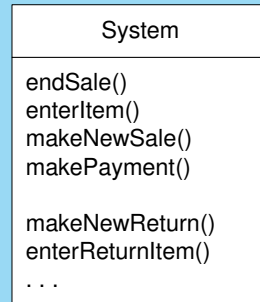
3. Related Patterns: GoF Command, Façade

Example: Controller (GRASP)



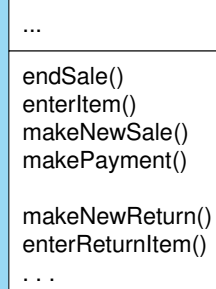
Example: Controller (GRASP)

System Interface



system operations
discovered during system
behavior analysis

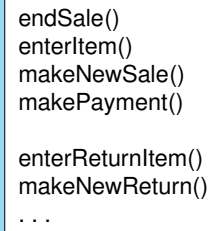
Register



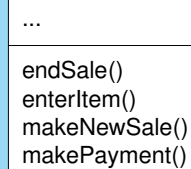
allocation of system
operations during design,
using one facade controller

**system facade
controller**

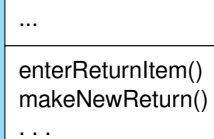
System



ProcessSale
Handler



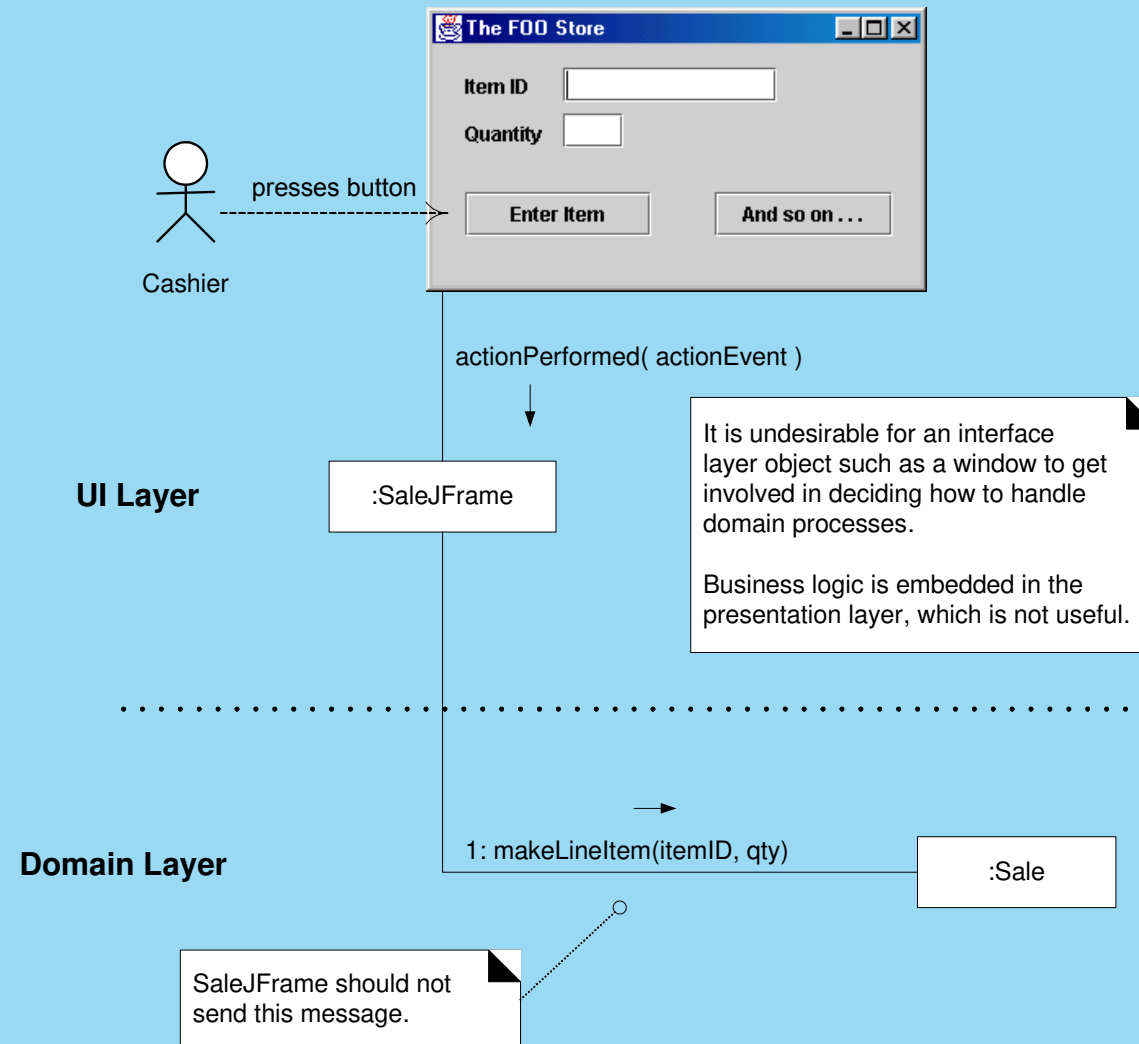
HandleReturns
Handler



allocation of system
operations during design,
using several use case
controllers

**use case / session
controllers**

Example: Controller “Anti Pattern” (GRASP)



High Cohesion (GRASP)

Def: Cohesion is a measure of how strongly related and focused the responsibilities of an element are.

1. Problem

- How to keep objects focused, understandable, and manageable, and as a side effect support Low Coupling?

2. Solution:

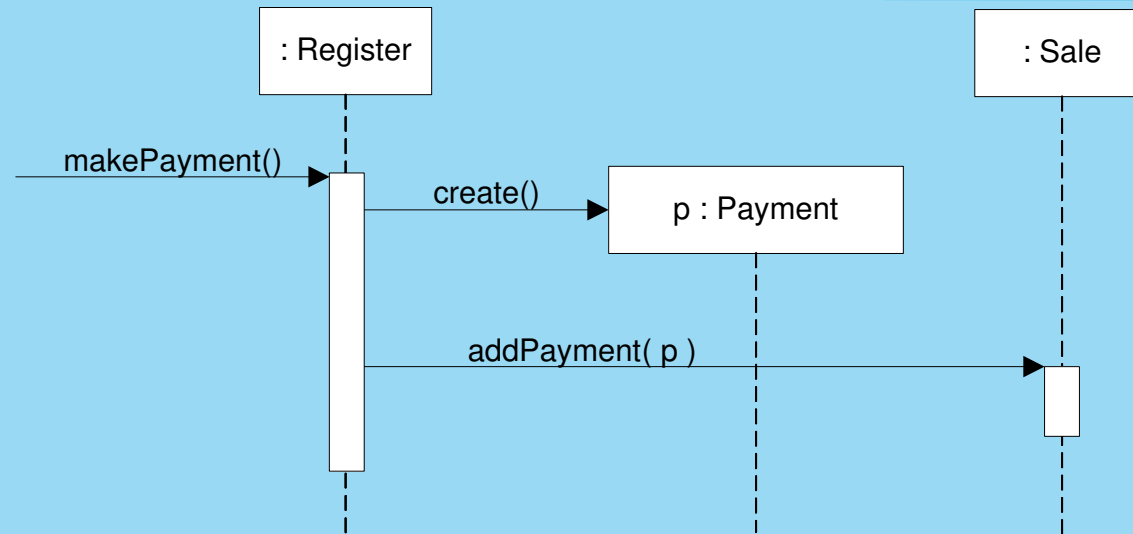
- Assign responsibilities so that cohesion remains high.

3. Benefits

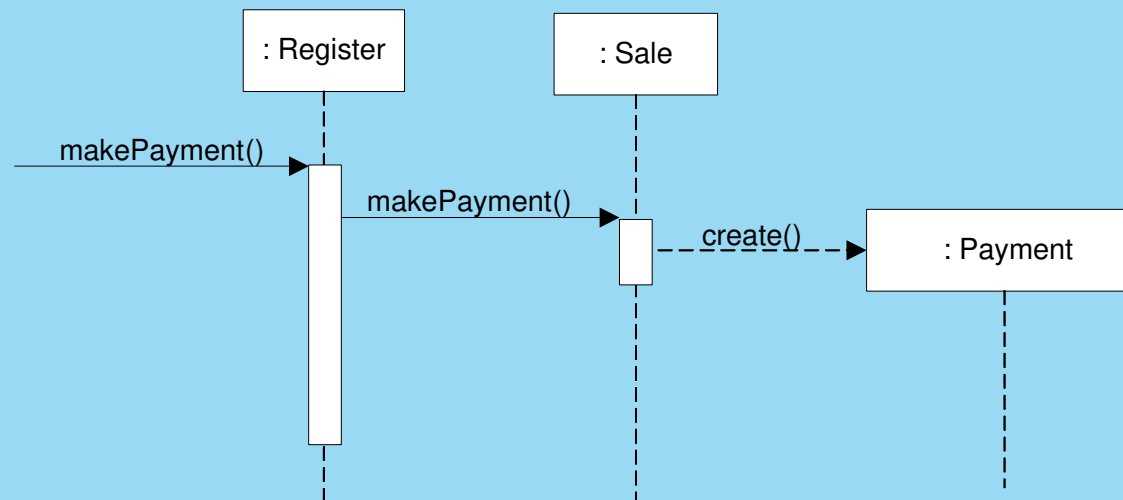
- Design of classes that are more independent
- Reduced impact of changes
- Simple to understand in isolation
- Easier to reuse

Example: High Cohesion (GRASP)

**Design
Alternative 1**



**Design
Alternative 2**

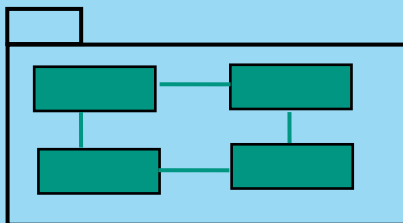


Relation between Low Coupling & High Cohesion

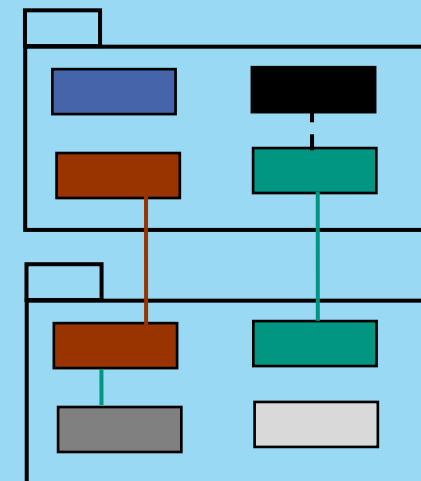
- Low coupling between software entities normally leads to a high cohesion inside the entities and vice versa
- The Yin and Yang of software development [Larman]
 - necessary to **reduce impact of changes** and **foster reuse**
 - classes with many dependencies have a high coupling
 - classes with many different responsibilities have low cohesion
- Driven by the idea of **separating concerns**



good



not so good

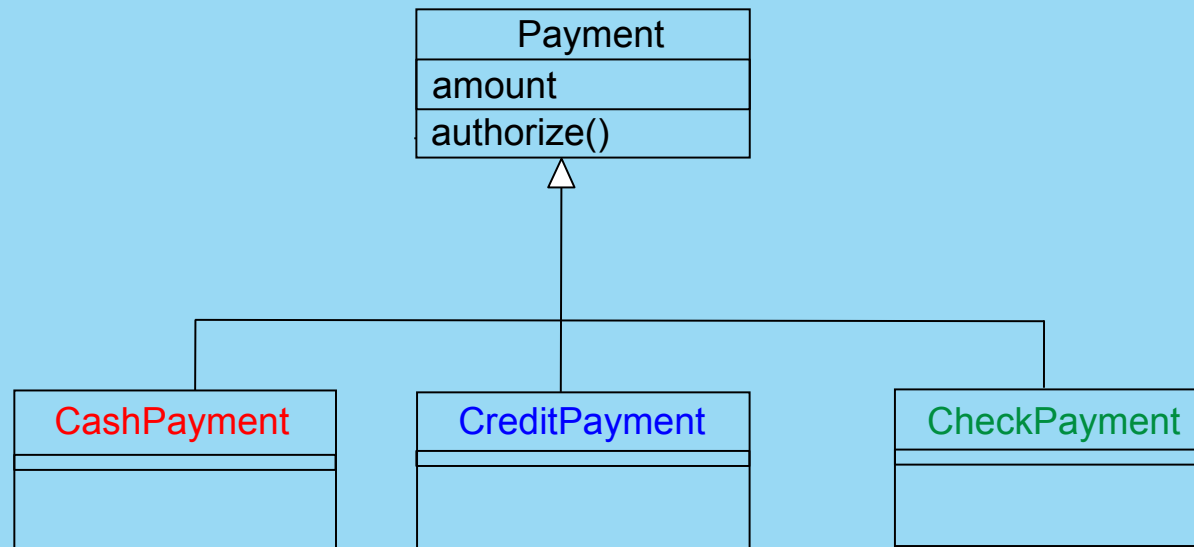


Motivation: Polymorphism (GRASP)

- **Question:** How to implement alternative behavior for a set of related objects that varies depending on the concrete object type?

Approach 1

Bad design!



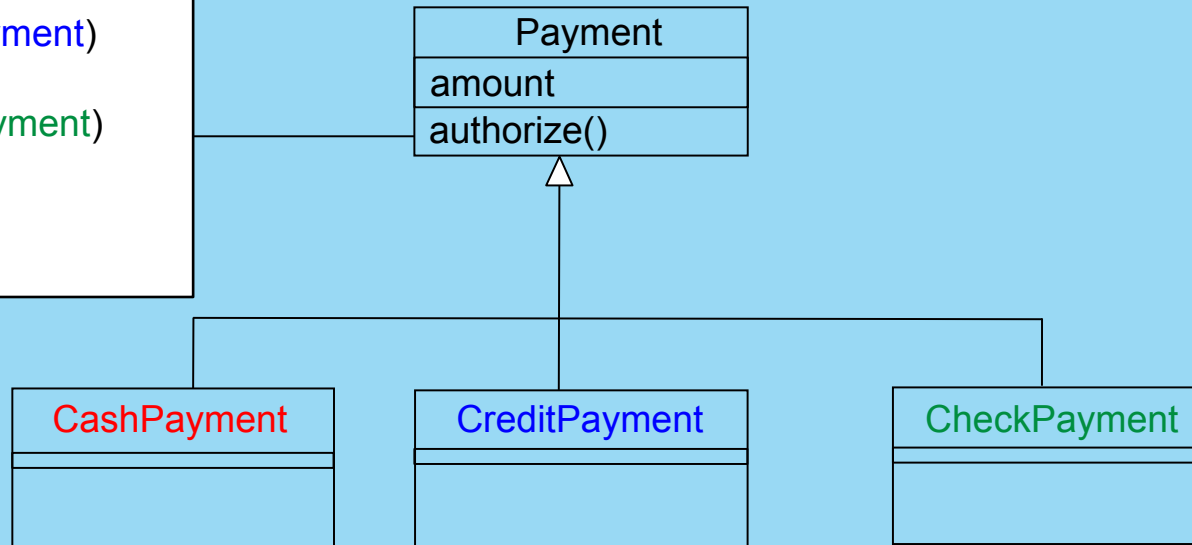
Motivation: Polymorphism (GRASP)

- **Question:** How to implement alternative behavior for a set of related objects that varies depending on the concrete object type?

Approach 1

```
authorize() {  
  if (this instanceof CashPayment)  
    ...  
  else if (this instanceof CreditPayment)  
    ...  
  else if (this instanceof CheckPayment)  
    ...  
}
```

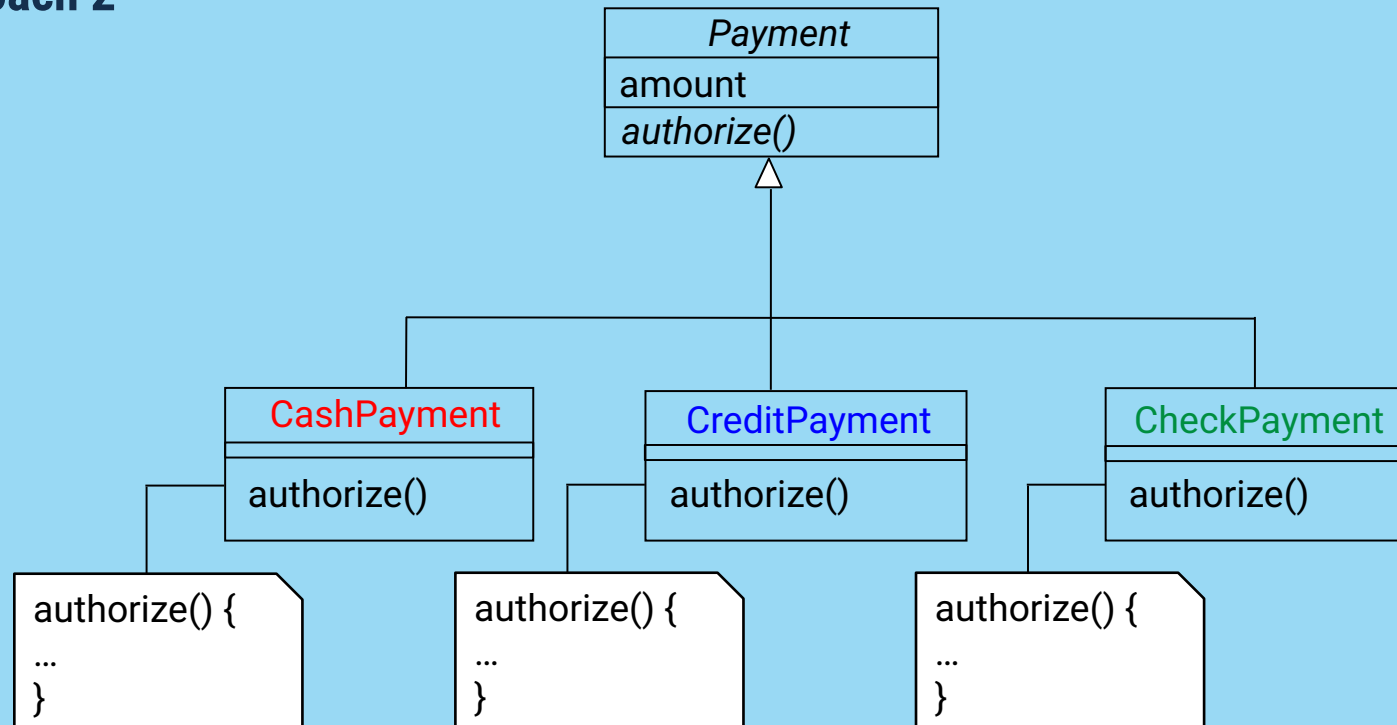
Bad design!



Motivation: Polymorphism (GRASP)

- **Question:** How to implement alternative behavior for a set of related objects that varies depending on the concrete object type?

Approach 2

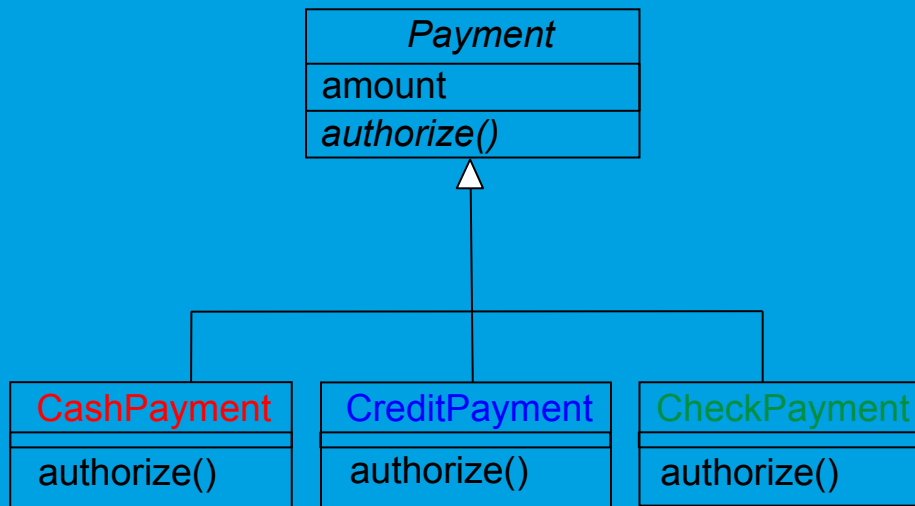


Good design

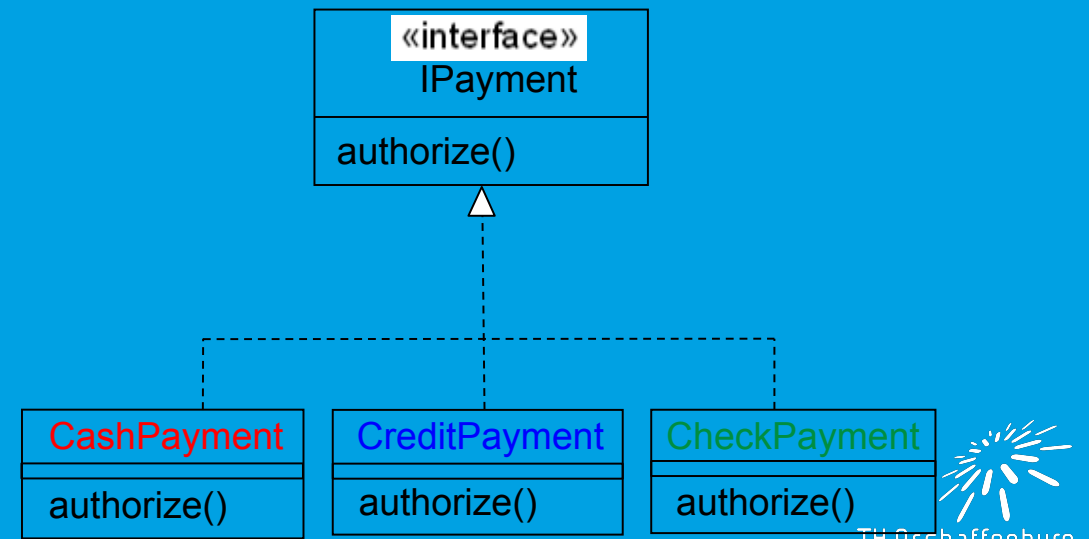
Polymorphism (GRASP)

Def (Polymorphism): Giving the same name to operations (methods) in different objects when the operations are similar or related. The different object types usually implement a **common interface** or are related in an implementation hierarchy with a **common superclass**.

Polymorphism realized
using **Inheritance**



Polymorphism realized
using **Interface**



Polymorphism (GRASP)

Def (Polymorphism): Giving the same name to operations (methods) in different objects when the operations are similar or related. The different object types usually implement a **common interface** or are related in an implementation hierarchy with a **common superclass**.

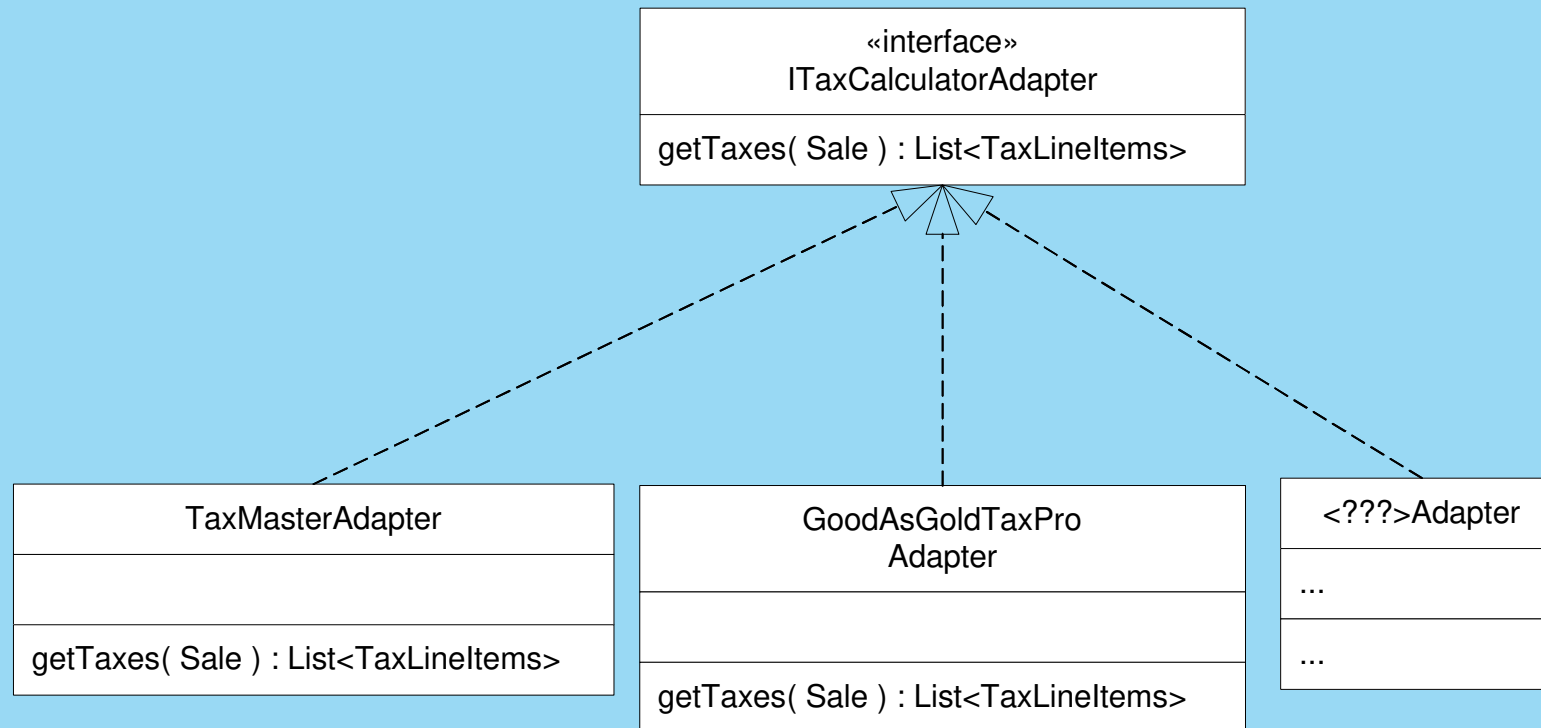
1. Problem

- How to implement alternative behavior for a set of related objects that varies depending on the concrete object type?

2. Solution

- Instead of assigning the responsibility for the behavior to a single object (respectively operation) and then using “if” statements to test for the type of the object (Approach 1 in our example),
- Use polymorphic operations and assign the responsibility for the behavior to the respective classes (types) for which the behavior varies (Approach 2).

Example: Polymorphism (GRASP)



By Polymorphism, multiple tax calculator adapters have their own similar, but varying behavior for adapting to different external tax calculators.

Motivation: Pure Fabrication (GRASP)

- What if no appropriate domain object exists for a given responsibility?
 - Invent an artificial class and assign cohesive responsibilities to it
 - Preserves high cohesion and low coupling
- **Example**
 - Support needed to save Sale instances in a relational database
 - There would be some justification to assign this responsibility to Sale
 - justified by the **Information Expert pattern**
 - however, this requires a relatively large number of supporting database-oriented operations
 - the Sale class would thus become incohesive
 - On the other hand, it is a general task for which many classes need support
- → Create a new class PersistentStorage (or EntityManager) to solve this
 - Belongs to the technical services layer (as it is reusable in other contexts)

Pure Fabrication (GRASP)

1. Problem

- What object should have the responsibility, when you do not want to violate High Cohesion and Low Coupling, or other goals, but solutions offered (for example) by the Information Expert pattern are not appropriate? Many situations in which assigning responsibilities only to classes corresponding to domain concepts leads to problems in terms of poor cohesion or coupling, or low reuse potential.

2. Solution

- Assign a highly cohesive set of responsibilities to an **artificial or convenience class** that does not represent a problem domain concept – something made up, to support high cohesion, low coupling, and reuse.

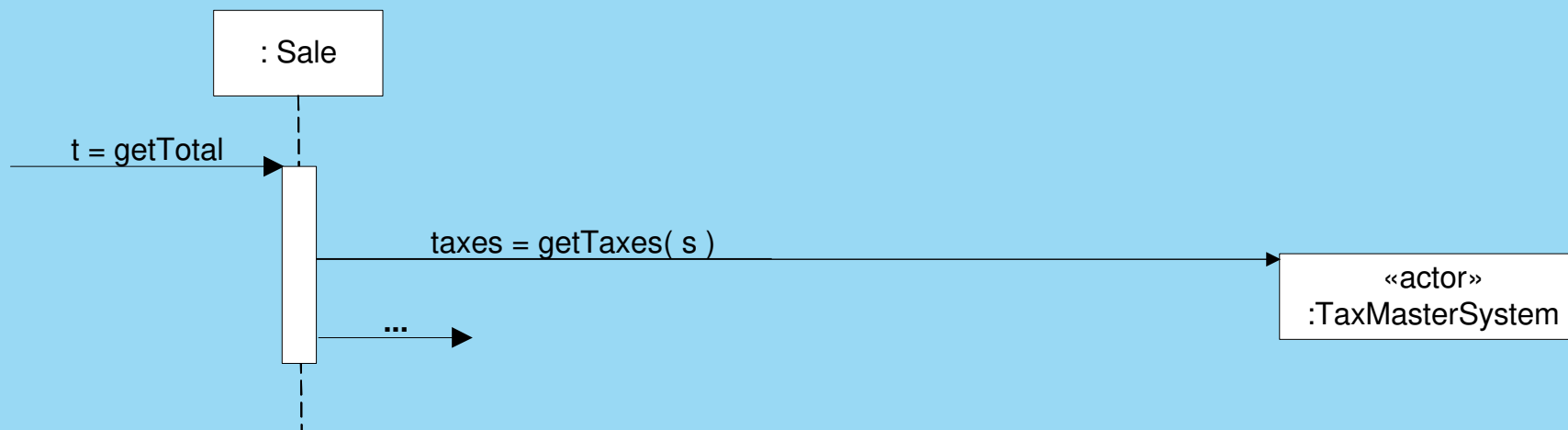
3. Related Patterns

- All GoF design patterns, such as Adapter, Command, Strategy, and so on, use Pure Fabrications.
- Generally, most design patterns use Pure Fabrications.

Indirection (GRASP)

- Problem

- Assigning a given responsibility to a considered target class (e.g., suggested by other GRASP patterns) would yield a high coupling or low cohesion?



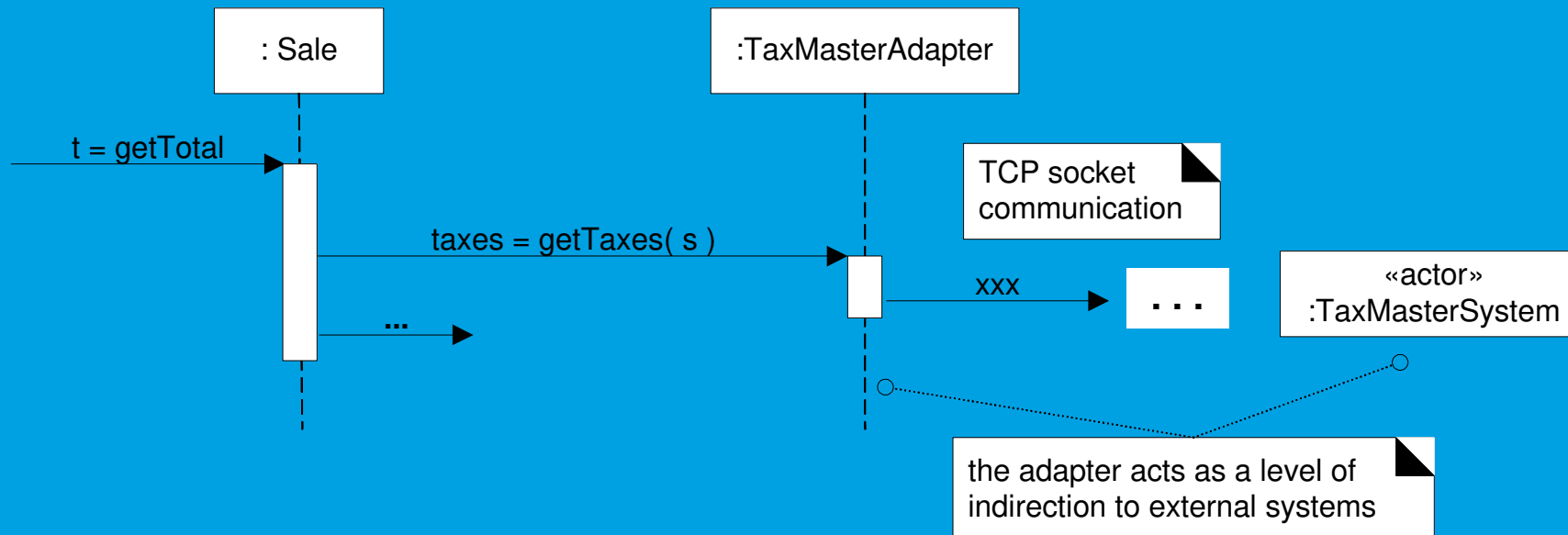
Indirection (GRASP)

■ Problem

- Assigning a given responsibility to a considered target class (e.g., suggested by other GRASP patterns) would yield a high coupling or low cohesion?

■ Solution

- Create an **adapter object** to mediate between the respective classes (a.k.a. **proxy**)



Indirection as Fundamental Principle in CS

- **All problems in computer science can be solved by another level of indirection...**
-- David Wheeler
- **"...except for the problem of too many levels of indirection."**
-- Kevlin Henney



Protected Variations (GRASP)

■ Problem

- How to design objects, components, and (sub)-systems so that the variations or instability in these elements does not have an undesirable impact on other software elements?

■ Solution

- Identify points of **predicted variation (instability)** and create a **stable interface** around them
- Interface is used in the broadest sense of an **access view**
- Most design principles and **patterns are mechanisms for protected variation**, including polymorphism, indirection, data encapsulation, and most of the GoF patterns

■ Essentially equivalent to the

- **Information Hiding Principle** (Parnas): “Only what is hidden can be changed without risk...”
- **Open-Closed Principle** (Mayer): *More on this later...*



Question

Which of these statements are true?

1. Low Coupling supports/fosters Protected Variation
2. Polymorphism supports/fosters Low Coupling
3. Indirection supports/fosters Low Coupling
4. Pure Fabrication supports/fosters High Cohesion

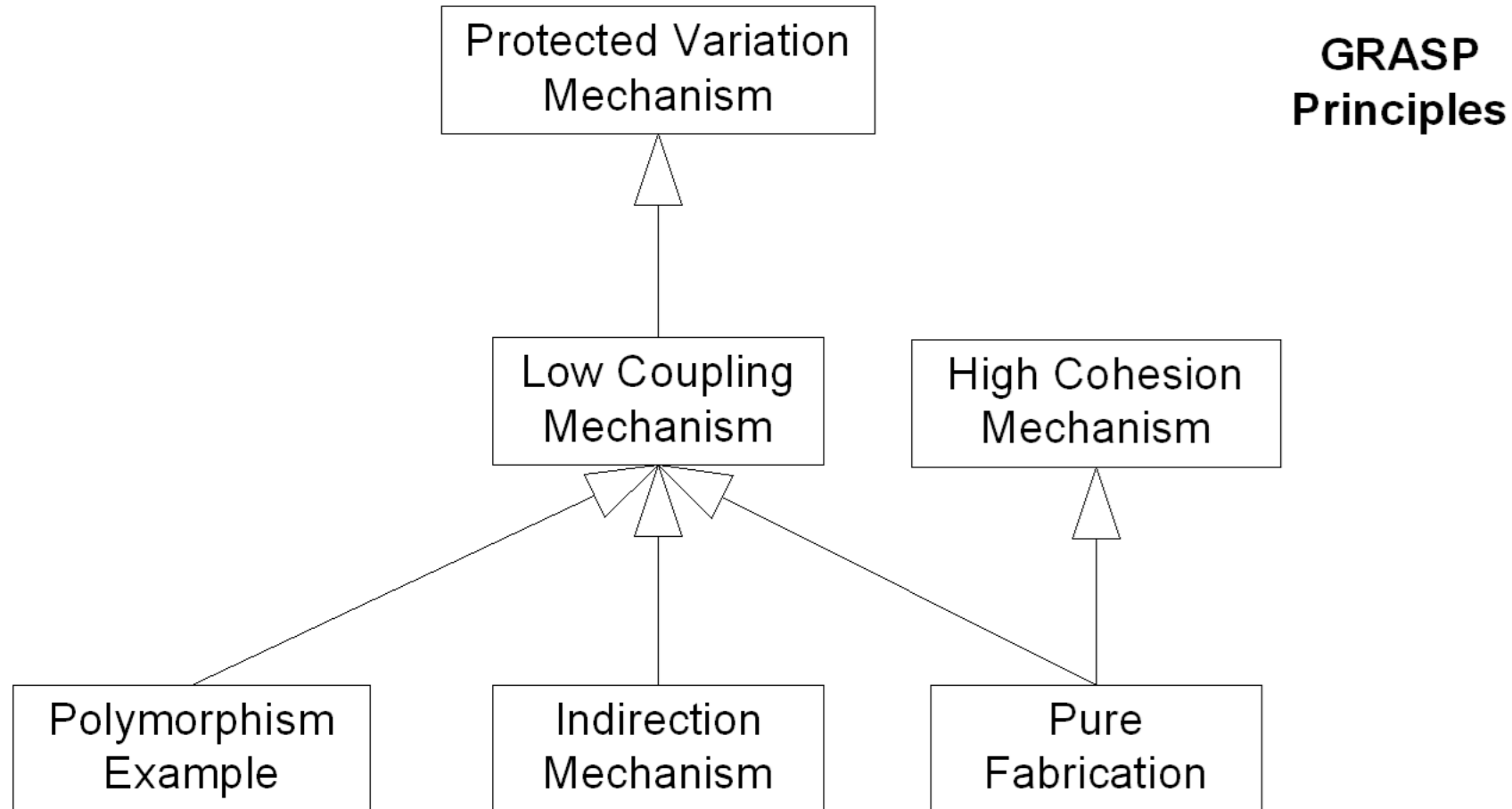
a) 1, 2, and 3

b) 1, 3, and 4

c) 2, 3, and 4

d) all

Relationships between GRASP Principles



Important

- In practice, iterative design should be kept light and short
 - Do not try to detail everything in a UML model upfront
 - focus on modeling the creative and difficult parts
 - try to move on quickly to code and test in each iteration
 - and update the diagrams accordingly

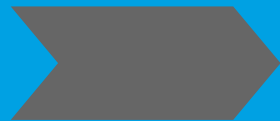
*“The cheapest, fastest, and most reliable (and most accurate, most secure, easiest to design, document, test and maintain) **components** of a computer system are those that aren’t there.”* -- Gordon Bell

Object Design (a.k.a. Object-Oriented Design)

Motivation for Good Object Design

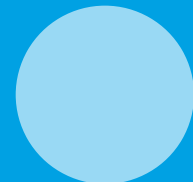
Responsibility-Driven Design (RDD)

GRASP Principles



Further Software Design Principles

Concluding Remarks



(Further) Software Design Principles

- **“SOLID”**
 - An acronym introduced by Michael Feathers for the "first five principles" of object-oriented programming and design
- Named by Robert C. Martin in the early 2000s [Martin]
 - **S**ingle Responsibility Principle
 - **O**pen/Closed Principle
 - **L**iskov's Substitution Principle
 - **I**nterface Segregation Principle
 - **D**ependency Inversion Principle

OO Programming and Design Principles [Martin]

Single Responsibility Principle (SRP)

- A class should have only a single main responsibility (i.e., only one potential change in the software's specification should be able to affect the class).

Open/Closed Principle (OCP)

- Software entities ... should be open for extension but closed for modification.

Liskov's Substitution Principle (LSP)

- Objects in a program should be replaceable with instances of their subclasses without altering the correctness of that program.

Interface Segregation Principle (ISP)

- Many client-specific interfaces are better than one general-purpose interface.

Dependency Inversion Principle (DIP)

- One should depend upon abstractions, not upon concretions.

DRY, KISS & YAGNI Principles

Don't repeat yourself (DRY)

- Every piece of knowledge must have a single, unambiguous, authoritative representation within a system.
- Avoid redundancy in code as it quickly becomes ugly to maintain.

Keep it short and simple (KISS)

- Every design should be as simple as possible but not simpler.
(originated in airplane design, planes must be repairable under combat conditions)



You ain't gonna need it (YAGNI)

- Only implement required features!
- Resist the temptation to write something that might become necessary in the future.

*"Make everything as **simple** as possible but not simpler"*

-- Albert Einstein

Object Design (a.k.a. Object-Oriented Design)

Motivation for Good Object Design

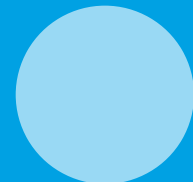
Responsibility-Driven Design (RDD)

GRASP Principles

Further Software Design Principles



Concluding Remarks



Take Away – OO Design

- The design of OO software is largely based on the **assignment of responsibilities** to software objects
 - GRASP (principles) encapsulate the basic rules
 - design should be done via modeling the dynamic interplay of objects with interaction diagrams
- The actual design phase should be kept short
 - move to coding and testing quickly



References (1/3)

- [Ambler04] S. Ambler
The Object Primer: Agile Model-Driven Development with UML 2.0
Cambridge University Press, 2004.
- [Coad92] P. Coad
Object-Oriented Patterns
Communications of the ACM, Vol. 35, Iss. 9, 1992
<http://portal.acm.org/citation.cfm?id=131006>
- [Endres/Rombach03] A. Endres, D. Rombach
A Handbook of Software and Systems Engineering
Addison-Wesley, 2003
- [Larman05] C. Larman
Applying UML and Patterns
Prentice Hall, 2005
- [Wirfs-Brock90] R. Wirfs-Brock et al.
Designing Object-Oriented Software
Prentice Hall, 1990

References (2/3)

- [PP99] A. Hunt and D. Thomas
The Pragmatic Programmer
Addison-Wesley, 1999
- [Martin] R. Martin “Uncle Bob”
Design Principles and Design Patterns
http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf
- [Meyer88] B. Meyer
Object-Oriented Software Construction
Prentice Hall, 1988.
- [Martin2008] Martin, R.C.:
Clean Code: A Handbook Of Agile Software Craftsmanship,
Prentice Hall, 2009.
- [Martin2002] Martin, R.C.:
Agile Software Development: Principles, Patterns, and Practices,
Prentice Hall, 2002.

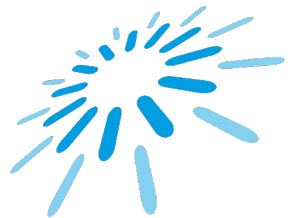
References (3/3)

- [Fowler1999] Fowler, M.:
Refactoring: Improving the Design of Existing Code,
Addison-Wesley, 1999.
- [Johnson1990] Johnson, R., Opdyke W.:
Refactoring: An aid in designing application frameworks and evolving object-oriented
systems.
In: Proceedings of Symposium on Object-Oriented Programming Emphasizing Practical
Applications (SOOPPA), 1990.
- [GOF1995] Gamma, E., Helm, R., Johnson, R., Vlissides, J.:
Design Patterns: Element of Reusable Object-Oriented Software,
Addison-Wesley, 1995.
- [Freeman2006] Freeman, E., Freeman, E.:
Head First – Design Patterns,
O'Reilly. 2004.



Until next time!

Software Architecture and Design Patterns
Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg
university of applied sciences